Dialga: A Family of Low-Latency Tweakable Block Ciphers using Multiple Linear Layers (Full Version) *

Subhadeep Banik[1], Tatsuya Ishikawa[2], Takanori Isobe[2], Ryoma Ito[2,3], Kazuhiko Minematsu[4], Kazuma Nakata[5], Mostafizar Rahman[6] and Kosei Sakamoto[2,7]

[1] University of Lugano, Lugano, Switzerland
subhadeep.banik@usi.ch
[2] The University of Osaka, Suita, Japan
t.ishikawa037@gmail.com, takanori.isobe@ist.osaka-u.ac.jp
[3] NICT, Koganei, Japan
itorym@nict.go.jp
[4] NEC, Kawasaki, Japan
k-minematsu@nec.com
[5] University of Hyogo, Kobe, Japan
marokazu670@gmail.com
[6] Kyoto University, Kyoto, Japan
mrahman454@gmail.com
[7] Mitsubishi Electric Corporation, Kamakura, Japan
sakamoto.kosei@dc.mitsubishielectric.co.jp

Abstract. In this paper, we propose Dialga, a family of low-latency tweakable block ciphers designed to support 128/256-bit tweaks and 256-bit keys. Dialga achieves significantly small latency by leveraging multiple novel strategies. These include the use of multiple linear layers with efficient cell permutations, which enhance security against differential and linear attacks with negligible hardware overhead. We also identify the optimal choice of S-boxes for these permutations using state-of-the-art evaluation methods by SAT, enabling us to further reduce the delay of the round function. Besides, we design a reflection tweakey schedule that ensures strong security in the related-tweak setting and allows for encryption and decryption without delay overhead, reducing the circuit area. We conducted comprehensive hardware benchmarks involving Dialga and other primitives. As a result, Dialga achieves nearly half the delay of QARMAv2, while achieving approximately a 40% reduction in area, with the same claimed security. We also demonstrate that Dialga enables an efficient low-latency TBC-based authenticated encryption instantiation: Flat-ΘCB based on Dialga compares favorably with AES-256-GCM in hardware, achieves substantially lower delay than AES-256-GCM.

Keywords: Ultra-low latency · Tweakable block cipher · Multiple linear layers · Memory encryption · Beyond 5G (B5G), Authenticated encryption

*Compared with the ToSC version [BII+25], we added variants for pointer authentication and memory-integrity applications and included a comparison of the low-latency TBC-based AE (Flat-ΘCB) mode with AES-GCM.

1 Introduction

1.1 Background

Low-latency primitive for latency-critical applications. Due to the recent technological progress on non-volatile memory devices and advancements in memory attack methods such as RowHammer [KDK+14] and RAMBleed [KGGY20], memory encryption receives significant attention. The security goals and the general mechanism for a memory encryption scheme differ by the target system architecture and the constraints. For example, memory encryption inside Trusted Execution Environment such as Intel's SGX [Gue16] ensures confidentiality and authenticity and replay protection, while a much simpler XTS [Dwo10] only ensures block-wise confidentiality (see Avanzi [Ava22] for classification). A low-latency cryptographic primitive is crucial for any type of memory encryption. A study of low-latency has been conducted for more than a decade: PRINCE, a 64-bit block cipher that explicitly aims at low-latency operation, was proposed at Asiacrypt 2012 [BCG+12]. It is acknowledged as one of the earliest low-latency primitive proposals. QARMA is a low-latency tweakable block cipher (TBC) [Ava17] whose design is based on Prince and has been deployed for Pointer Authentication, an integrity protection mechanism of code pointers inside ARM CPUs. Security of them have been extensively studied (e.g., [JNP+14, SBY+14, Din15, DP15, GR16, LHW19, CXTQ23]), and since then, design and analysis of low-latency primitives have become a hot topic [BEK+20, BIL+21, LSW22, TISI23, LMMR21, BDBN23, WNL+23, RS22, BDD+23a, CGL+23, WBD+24].

Another application where low-latency primitive will play a key role is 6G communication, which will be used in autonomous vehicles, remote surgery, virtual and augmented reality (VR/AR), industrial automation, and smart cities[1]. According to Nokia's whitepaper[2], to support a diverse range of applications and devices, sub-nanosecond latency for encryption is desired, allowing even the most demanding tasks to be performed in real-time, meeting the high-performance demands of next-generation technologies.

QARMAv2 and long-tweak TBCs. At ToSC 2023, QARMAv2 has been introduced by Avanzi et al. [ABD+23] as a family of low-latency 64/128-bit TBCs. As well as the initial version, QARMAv2 aims at low-latency operation. However, its profile is extended by supporting up to 256-bit key and 256-bit tweak, depending on the block size. Notably, it contains a member supporting tweaks longer than the block size. The paper [ABD+23] mentions that such a long(er) tweak would be usable in future-proofing memory encryption and that already in the current architecture, say, a tweak may need to absorb 48- to 52-bit address, 56-bit counter, and 32-bit process domain ID, which exceeds 128 bits in total. Moreover, a type of memory/storage encryption similar to XTS uses a tweak to contain metadata and addresses, and the amount of metadata will grow for future systems, e.g., to support larger memory, advanced functionalities, and compatibility. We emphasize that invertibility is needed for XTS-type encryption since it is a tweakable variant of ECB mode. Hence, we cannot employ non-invertible designs, such as Orthros [BIL+21] and Gleeok [ABC+24]. A TBC-based authenticated encryption (AE), based on the seminal ΘCB [KR11], is also proposed in the context of tree-based memory encryption scheme [IMO+22], where the latency of TBC inside the AE often turns out to be the bottleneck.

In a broader context, a TBC supporting a tweak longer than the block – let us call it a long-tweak TBC – is actively studied for its usability in highly-efficient and highly-secure mode designs. For example, such a long-tweak TBC enables "beyond-full-bit"[3] security for AE. Naito et al. [NSS20] proposed the first beyond-full-bit-secure AE mode at Eurocrypt

[1] https://www.6g-forum.com/
[2] https://www.nokia.com/about-us/newsroom/articles/nokias-vision-for-the-6g-era/
[3] Namely, s-bit security for some $s > n$, using an n-bit block primitive.

2020 based on a long-tweak TBC together with a proposal of new 64-bit block, 256-bit tweak variant of SKINNY [BJK+16], SKINNYe.

A long-tweak TBC enables AE with various advanced features, such as Threshold-implementation (TI)-friendly AEs [NSS20, NSS22], efficient leakage-resilient AEs [SPS+22, PSS24], and high multi-user security [CJPS24]. Consequently, a long-tweak TBC is promising, while there are few studies on efficient designs. In the case of very small block size, two application-specific long-tweak TBCs have been proposed, namely BipBip (24-bit block and 40-bit tweak) [BDD+23a] and SCARF [CGL+23](10-bit tweak and 48-bit tweak). Some long-tweak TBCs have been cryptanalyzed [QDW+22, WBD+24]. To the best of our knowledge, QARMAv2 is the sole TBC proposal aiming at low latency operation while supporting long tweaks and conventional block sizes. Since its proposal, it has received significant attention from cryptanalysts, with integral distinguishers subsequently improved in [HGSE24, HT24], while fault attacks have shown susceptibility, significantly reducing the effective key space for variants of QARMAv2 [SCS23]. Additionally, a truncated differential attack has been mounted on QARMAv2-64 [AKM+24]. An invariant of the round function of QARMAv2-64 is reported [Bey23]. The current research status suggests that the efficient design of long-tweak TBC is a challenging task and is relevant in both theory and practice. Applications of such TBC are not only limited to memory encryption/6G applications.

Our Contributions. QARMAv2 is a well-designed low-latency TBC supporting long (256-bit) tweak and long key. However, its latency is not ultimately small. When compared with the non-invertible alternative, Gleeok [ABC+24], published at TCHES 2024, the latency of QARMAv2's encryption is slower by a factor of three. Gleeok has a parallel structure (a sum of three permutations) which enables quite low latency; however, losing invertibility limits usability as mentioned above. With this situation in mind, we aim at designing a TBC supporting long tweak and long key as QARMAv2, i.e., 256-bit tweak and 256-bit key (at maximum) that provides competitive latency performance to the state-of-the-art low-latency PRF, Gleeok. Additionally, we focus on minimizing area consumption to facilitate deployment in diverse devices and applications, including unified encryption/decryption circuits. As a result, we introduce Dialga, a family of low-latency TBCs. A member of Dialga supports 256-bit tweak and 256-bit key which is compatible with QARMAv2, yet achieves significantly smaller latency than QARMAv2, keeping comparable (or even smaller) circuit size. Supporting 256-bit key will be relevant in the context of post-quantum cryptography and will be required for future latency-critical applications (see Introduction of [ABC+24] for further discussions).

1.2 Design of Dialga

To design Dialga, we leverage novel strategies built on top of block cipher Midori [BBI+15] for low-latency purposes. These strategies are summarized as follows.

- Multiple Linear Layers: As unrolled implementations are a primary target for low-latency ciphers, using the same function for each round is unnecessary. Moreover, cell permutations in hardware can be implemented with negligible overhead, allowing for varied permutations per round without significant overhead. We introduce multiple cell permutations into the linear layers of a Midori-like cipher. By employing an efficient filtering approach for a large number of candidates, we identify a class of multiple cell permutations that significantly enhance security, particularly against differential and linear attacks, compared to Midori. In the Midori-like cipher, the single permutation used in Midori is optimal from the perspective of active S-boxes (AS) [ABI+18]. Whether multiple permutations can improve upon this remains a non-trivial question. This paper presents the first case study demonstrating that the use of multiple permutations can offer significant advantages over a (optimal) single

permutation, without introducing additional overhead in the unrolled implementation. This improvement enables a reduction in the number of rounds, thereby minimizing the overall delay in latency-critical designs.[4]

- Optimal Choice of S-box with Multiple Linear Layer: We investigate optimal S-box choices for the identified multiple cell permutations. Specifically, by incorporating bit-wise differential/linear transitions of S-boxes, we obtain exact differential/linear characteristic probabilities to find optimal combinations by leveraging state-of-the-art efficient evaluation methods by SAT solvers. This allows us to further reduce the delay of round function compared to Midori, while improving the security. While the use of different round functions is not a new concept, we demonstrate that our novel approach—a structure with multiple linear layers and optimally selected components—achieves the desired security level with a significantly reduced delay, specifically achieving half the delay of the state-of-the-art cipher, QARMAv2.

- Reflection Tweakey Schedule: In TBCs, achieving security against differential attacks in the related-tweak setting is challenging due to full adversary control over tweaks. We design a tweakey schedule for Dialga that ensures strong security and efficient implementation. By incorporating multiple cell shuffles into the tweakey schedule, we reduce the required number of rounds to guarantee security against differential attacks in the related-tweak setting. The reflection tweakey schedule, with two inversely related parts, allows encryption and decryption without delay overhead, reducing area by removing the second part of the circuit.

1.3 Implementation Results

We evaluate Dialga in an unrolled implementation and compare it to other ciphers. Dialga exhibits nearly half the delay of QARMAv2 and achieves approximately a 40% reduction in area and latency while claiming equivalent security level. Since we use Midori as a starting point, our objective was twofold a) to extend the security level to 256 bits and b) to securely incorporate a tweak in the design, and at the same time provide a total end-to-end delay which is equal or competitive wrt Midori. Hardware simulation results tabulated in Table 18 show that we were successful to a large extent in this goal.

Further in Section 6, we present a benchmark result of low-latency TBC-based AE (Flat-ΘCB) mode [IMO+22] using Dialga. This AE mode is based on ΘCB [KR11] and is proposed to be used in SGX-type memory encryption. Flat-ΘCB achieves theoretical minimum latency for both encryption and decryption, namely a single TBC call in a fully parallel implementation (which improves ΘCB as its decryption generally needs two TBC calls). The implementation results show that the circuit for the mode can be constructed with relatively low overhead over and above the block cipher circuit both in terms of additional area and circuit latency and can effectively process a block of incoming data per clock cycle, which can be increased further by having multiple parallel instances of the circuit. We also introduce 12-round variants, Dialga-128-PA and Dialga-256-PA, for pointer authentication and memory-integrity applications, following the motivation of QARMAv2; these variants are intended for truncated-tag settings where low latency is particularly important.

Additionally, in the Appendix, we present a unified circuit for encryption and decryption. Dialga uses 4 different round functions that are non-involutive, and so the unified circuit for Enc/Dec is not as straightforward or efficient as PRINCE or QARMA. However, we

[4] PRINCE [BCG+12] and QARMA [Ava17] utilize different permutations; however, their purpose is implementation(area)-oriented, specifically for integrating encryption and decryption circuits. It remains unclear whether a single permutation or multiple permutations are optimal from a security standpoint for these schemes.

show that if we optimize for area, the circuit can be realized with less than a 30% increase in area.

Finally, a description of round-based, 2/4 round unrolled, and masked circuits is presented in Appendices G and H.

Remark. Independently of our work, Hu et al. [HKPT24] proposed uKNIT, a design framework for ultra-low-latency block ciphers that breaks the traditional round-alignment by allowing each round to be entirely different. While uKNIT relies on automated search algorithms to instantiate heterogeneous rounds, Dialga specifically leverages multiple alternating linear layers with efficient cell permutations to optimize the latency-security trade-off. Both works independently demonstrate the significant potential of non-uniform round structures in the design of low-latency cryptography.

2 Specification

Dialga is a family of TBCs with a 128-bit block, 256-bit key, and t-bit tweak, denoted Dialga-t where $t \in \{128, 256\}$. Both Dialga-128 and Dialga-256 have the round-reduced variants depending on the security claim, as explained in Section 2.3, denoted by Dialga-128† and Dialga-256†, respectively. The number of rounds for Dialga-128 and Dialga-256 are both 20 rounds. The number of rounds for Dialga-128† and Dialga-256† are both 16 rounds.

The data processing and tweak update during encryption and decryption of all variants of Dialga proceeds with the following 4×4 array:

$$S = \begin{bmatrix} s_0 & s_4 & s_8 & s_{12} \\ s_1 & s_5 & s_9 & s_{13} \\ s_2 & s_6 & s_{10} & s_{14} \\ s_3 & s_7 & s_{11} & s_{15} \end{bmatrix},$$

where $s_i \in \{0,1\}^8$ for $0 \le i \le 15$.

2.1 The Round Function

The round function of Dialga is applied to both the data processing and tweakey scheduling. Its structure is based on a Substitution Permutation Network (SPN) as expressed

$$R_i = \mathsf{MatrixMul} \circ \mathsf{Permutation} \circ \mathsf{SubCell},$$

The detailed descriptions of MatrixMul, Permutation, and SubCell are given as follows.

SubCell. This operation is the same as Subcell in Midori-128. Four kinds of 8-bit S-boxes SSb_0, SSb_1, SSb_2, and SSb_3 are applied to the internal state in parallel as follows:

$$s_i \leftarrow \mathsf{SSb}_{i \bmod 4}(s_i).$$

SSb_i consists of two 4-bit S-box Sb_0 in parallel with putting a bit permutation $P_{b,i}$ and its inverse $P_{b,i}^{-1}$ into the input and output, respectively. Thus, SSb_i is described as

$$\mathsf{SSb}_{i \bmod 4} = P^{-1}_{b,(i \bmod 4)} \circ (\mathsf{Sb}_0 || \mathsf{Sb}_0) \circ P_{b,(i \bmod 4)},$$

where $\mathsf{Sb}_0||\mathsf{Sb}_0$ denotes a parallel application of Sb_0. Tables 1 and 2 show a 4-bit S-box Sb_0 and each bit permutation, respectively. Figure 1 illustrates SSb_0, SSb_1, SSb_2, and SSb_3.

Table 1: 4-bit S-box Sb_0 in hexadecimal form.

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$\mathrm{Sb}_0(x)$	c	a	d	3	e	b	f	7	8	9	1	5	0	2	4	6

Table 2: Bit permutations: $P_{b,0}$, $P_{b,1}$, $P_{b,2}$, and $P_{b,3}$.

x	0	1	2	3	4	5	6	7	x	0	1	2	3	4	5	6	7
$P_{b,0}(x)$	4	1	6	3	0	5	2	7	$P_{b,1}(x)$	1	6	7	0	5	2	3	4
$P_{b,2}(x)$	2	3	4	1	6	7	0	5	$P_{b,3}(x)$	7	4	1	2	3	0	5	6

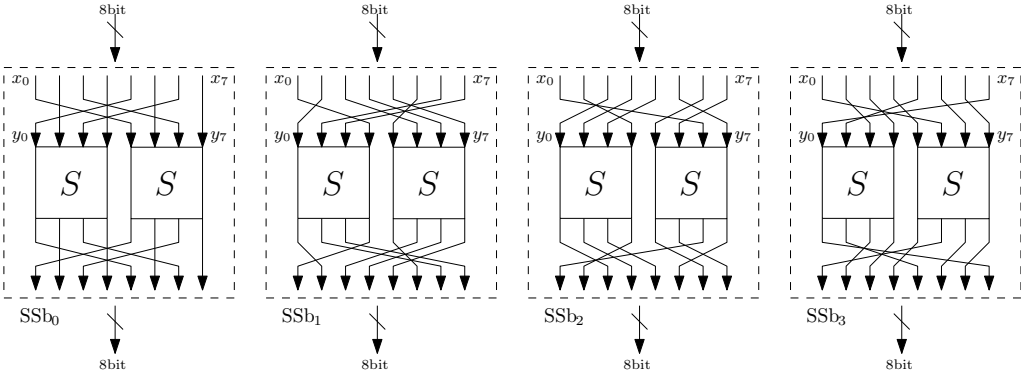

Figure 1: SSb_x

Table 3: Byte-wise permutations: π_0, π_1, π_2, and π_3.

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\pi_0(x)$	7	0	13	10	5	2	15	8	4	3	14	9	6	1	12	11
$\pi_1(x)$	13	0	10	7	11	6	12	1	2	15	5	8	4	9	3	14
$\pi_2(x)$	7	13	10	0	6	12	11	1	5	15	8	2	4	14	9	3
$\pi_3(x)$	13	8	6	3	14	11	5	0	12	9	7	2	15	10	4	1

Permutation. Both variants of Dialga use four byte-wise permutations depending on the round number as follows:

$$s_i \leftarrow s_{\pi_{(r-1 \bmod 4)}(i)},$$

where r denotes the round number. Table 3 shows byte-wise permutation π_0, π_1, π_2, and π_3.

MatrixMul. 4×4 binary matrix updates S as follows:

$$\begin{pmatrix} s_{4\cdot i} \\ s_{4\cdot i+1} \\ s_{4\cdot i+2} \\ s_{4\cdot i+3} \end{pmatrix} \leftarrow M \cdot \begin{pmatrix} s_{4\cdot i} \\ s_{4\cdot i+1} \\ s_{4\cdot i+2} \\ s_{4\cdot i+3} \end{pmatrix} = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} s_{4\cdot i} \\ s_{4\cdot i+1} \\ s_{4\cdot i+2} \\ s_{4\cdot i+3} \end{pmatrix}.$$

2.2 General Construction of Dialga

The encryption procedure of both variants of Dialga can be split into three parts R_f, R_m, and R_b as follows:

$$\mathsf{Enc} = R_b \circ R_m \circ R_f.$$

Table 4: The byte-wise permutation π_m in hexadecimal form.

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$\pi_m(x)$	0	a	5	f	e	4	b	1	9	3	c	6	7	d	2	8

As mentioned before, the data processing and tweakey scheduling use the round function R_i and its inverse R_i^{-1} while an additional byte-wise permutation π_m, which is identical to the one used in Midori as shown in Table 4, is applied into R_b. We denote MS as the application of π_m to the internal state. The detailed specifications of R_f, R_m, and R_b depend on the tweak size t.

Besides, in R_f, R_m, and R_b, we simultaneously add the round constants shown in Table 5 to the internal state when XORing the tweakey and the key to the internal state of the data processing. We use the decimal part of $\pi = 3.1415926\cdots$ as the round constants similar to PRINCE.

In the following explanations, let S_d and S_t be the internal states of the data processing and tweak scheduling, respectively. A 256-bit key K is divided into two 128-bit keys K_0 and K_1, i.e., $K = K_0||K_1$. A 256-bit tweak is divided into two 128-bit tweaks, i.e., $T = T_0||T_1$ and updated independently. Then, we denote the internal state of each tweak scheduling as S_{t_0} and S_{t_1}.

Table 5: Round constants in hexadecimal form.

c	Round constant	c	Round constant
c_0^f	0x243f6a8885a308d313198a2e03707344	c_0^m	0x9c30d5392af26013c5d1b023286085f0
c_1^f	0xa4093822299f31d0082efa98ec4e6c89	c_1^m	0xca417918b8db38ef8e79dcb0603a180e
c_2^f	0x452821e638d01377be5466cf34e90c6c	c_0^b	0x6c9e0e8bb01e8a3ed71577c1bd314b27
c_3^f	0xc0ac29b7c97c50dd3f84d5b5b5470917	c_1^b	0x78af2fda55605c60e65525f3aa55ab94
c_4^f	0x9216d5d98979fb1bd1310ba698dfb5ac	c_2^b	0x5748986263e8144055ca396a2aab10b6
c_5^f	0x2ffd72dbd01adfb7b8e1afed6a267e96	c_3^b	0xb4cc5c341141e8cea15486af7c72e993
c_6^f	0xba7c9045f12c7f9924a19947b3916cf7	c_4^b	0xb3ee1411636fbc2a2ba9c55d741831f6
c_7^f	0x0801f2e2858efc16636920d871574e69	c_5^b	0xce5c3e169b87931eafd6ba336c24cf5c
c_8^f	0xa458fea3f4933d7e0d95748f728eb658	c_6^b	0x7a325381289586773b8f48986b4bb9af
c_9^f	0x718bcd5882154aee7b54a41dc25a59b5	c_7^b	0xc4bfe81b6628219361d809ccfb21a991

2.2.1 Dialga-128

The total number of rounds in the data processing and tweakey scheduling are denoted r_d and r_t, respectively. As illustrated in Figures 2 and 3, Dialga-128 has $r_d = 16/20$ and $r_t = 8/10$, where the total number of rounds is set to 16 and 20 with the data complexity up to 2^{80} and 2^{128}, respectively, as discussed later in Section 2.3.

The specification of R_f, R_m, and R_b are as follows.

R_f R_f has an iterated construction that one iteration consists of two rounds of the data processing and one tweakey update. Before the first iteration, the tweak T and $K_0 \oplus K_1$ are XORed with the plaintext. Then, 5 and 4 iterations are applied in R_f for Dialga-128 and Dialga-128†, respectively. One iteration of R_f updates S_d and S_t as follows:

$$R_{f_1}(S_d, S_t, K) = \begin{cases} \text{if } i \text{ is the first iteration:} \\ \quad S_d \leftarrow R_{(2i-1)\%4}\big(R_{(2i-2)\%4}(S_d) \oplus K_{i\%2} \oplus c^f_{2(i-1)}\big) \\ \qquad \oplus S_t \oplus K_{(i-1)\%2} \oplus c^f_{2i-1}, \\ \quad S_t \leftarrow S_t \oplus K_{(i-1)\%2} \\ \\ \text{otherwise:} \\ \quad S_d \leftarrow R_{(2i-1)\%4}\big(R_{(2i-2)\%4}(S_d) \oplus K_{i\%2} \oplus c^f_{2(i-1)}\big) \\ \qquad \oplus R_{(i-1)\%4}(S_t) \oplus K_{(i-1)\%2} \oplus c^f_{2i-1}, \\ \quad S_t \leftarrow R_{(i-1)\%4}(S_t) \oplus K_{(i-1)\%2} \end{cases}$$

where i denotes the number of iterations regarding R_{f_1}. The range of i is $1 \leq i \leq 5$ and $1 \leq i \leq 4$ for Dialga-128 and Dialga-128†, respectively. Therefore, R_f is defined as

$$R_f(P, T, K) = R_{f_1}(P \oplus T \oplus K_0 \oplus K_1, T, K)^\alpha,$$

where α is 5 and 4 for Dialga-128 and Dialga-128†, respectively.

R_m R_m consists of two rounds of the data processing and one tweakey update with SubCell and its inverse SubCell^{-1} as follows:

$$R_m(S_d, S_t, K) = \begin{cases} S_d \leftarrow R_{(2\alpha+1)\%4}\big(R_{2\alpha\%4}(S_d) \oplus K_{(\alpha-1)\%2} \oplus c^m_0 \oplus \mathsf{SubCell}(S_t)\big) \\ \qquad \oplus MS\Big(R^{-1}_{(\alpha-1)\%4}\big(S_t \oplus K_{(\alpha-1)\%2}\big)\Big) \oplus c^m_1, \\ S_t \leftarrow R^{-1}_{(\alpha-1)\%4}\big(S_t \oplus K_{(\alpha-1)\%2}\big) \end{cases}$$

R_b R_b has an iterated construction that one iteration consists of two rounds of data processing and one tweakey update. One iteration of R_b updates S_d and S_t as follows:

$$R_{b_1}(S_d, S_t, K) = \begin{cases} \text{if } i \text{ is the last iteration:} \\ \quad S_d \leftarrow R_{(2(\alpha+i)+1)\%4}\big(R_{2(\alpha+i)\%4}(S_d) \oplus K_{(\alpha-i-1)\%2} \oplus c^b_{2(i-1)}\big) \\ \qquad \oplus MS\big(S_t \oplus K_{(\alpha-i-1)\%2}\big) \oplus c^b_{2i-1}, \\ \quad S_t \leftarrow S_t \oplus K_{(\alpha-i-1)\%2} \\ \\ \text{otherwise:} \\ \quad S_d \leftarrow R_{(2(\alpha+i)+1)\%4}\big(R_{2(\alpha+i)\%4}(S_d) \oplus K_{(\alpha-i-1)\%2} \oplus c^b_{2(i-1)}\big) \\ \qquad \oplus MS\Big(R^{-1}_{(\alpha-i-1)\%4}\big(S_t \oplus K_{(\alpha-i-1)\%2}\big)\Big) \oplus c^b_{2i-1}, \\ \quad S_t \leftarrow R^{-1}_{(\alpha-i-1)\%4}\big(S_t \oplus K_{(\alpha-i-1)\%2}\big) \end{cases}$$

where i denotes the number of iterations regarding R_{b_1}. The range of i is $1 \leq i \leq 4$ and $1 \leq i \leq 3$ for Dialga-128 and Dialga-128†, respectively. In R_b, one SubCell is applied, and then $K_0 \oplus K_1$ is XORed with S_d to generate the ciphertext after 3 and 4 iterations of R_{b_1} in Dialga-128 and Dialga-128†, respectively. Therefore, R_b is defined as

$$R_b(S_d, S_t, K) = \mathsf{SubCell}(R_{b_1}(S_d, S_t, K)^\beta) \oplus K_0 \oplus K_1,$$

where β denotes the number of iterations in R_b depending on the claimed security, i.e., β will be 4 and 3 for Dialga-128 and Dialga-128†, respectively.

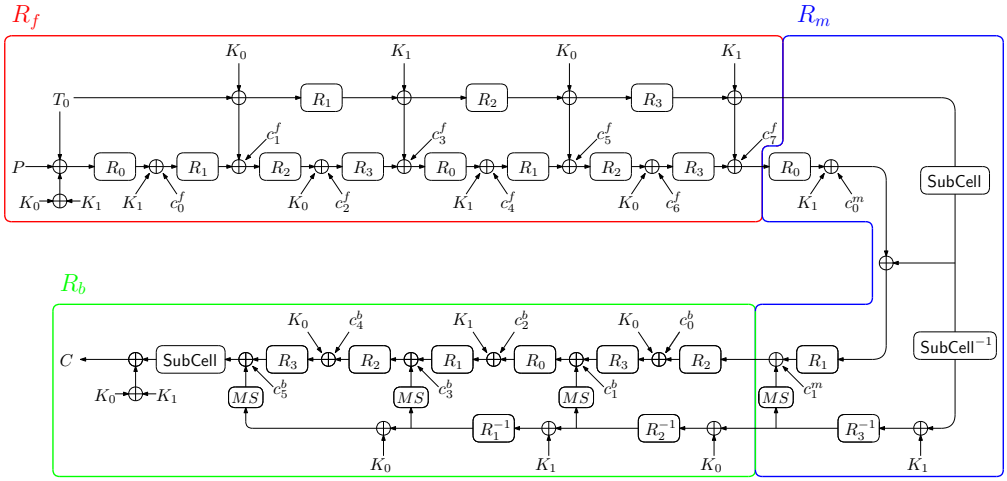

Figure 2: Overview of Dialga-128† (r_d=16).

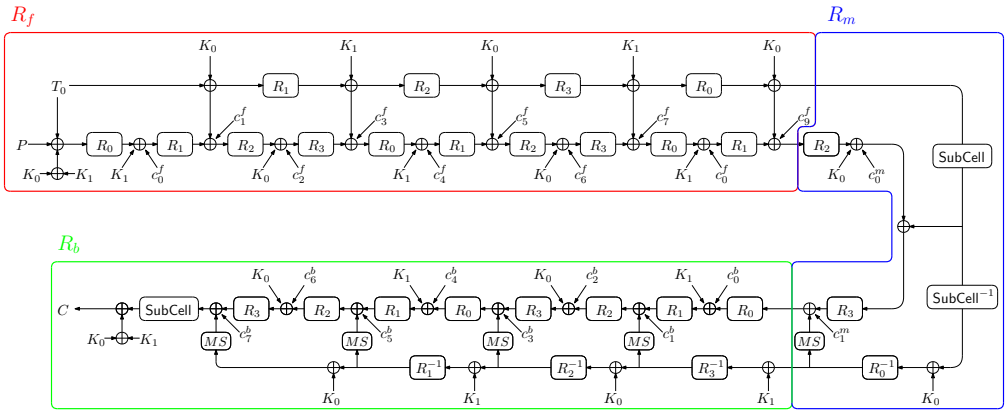

Figure 3: Overview of Dialga-128 (r_d=20).

2.2.2 Dialga-256

The total number of rounds in the data processing and tweakey scheduling are denoted r_d, (r_{t_0}, r_{t_1}), respectively. As illustrated in Figures 4 and 5, Dialga-256 has $r_d = 16/20$, $r_{t_0} = 8/10$, and $r_{t_1} = 10/8$, where the total number of rounds is set to 16 and 20 with the data complexity up to 2^{80} and 2^{128}, respectively, as discussed later in Section 2.3. The specification of R_f, R_m, and R_b are as follows.

R_f R_f has an iterated construction that one iteration consists of two rounds of data processing and one tweakey update for S_{t_0} and S_{t_1}. Before the first iteration, the tweak T_0 and $K_0 \oplus K_1$ are XORed with the plaintext. Then, 4 and 5 iterations are applied in R_f for Dialga-256 and Dialga-256†, respectively. One iteration of R_f updates S_d, S_{t_0}, and S_{t_1} as follows:

$$R_{f_1}(S_d, S_t, K) = \begin{cases} \text{if } i \text{ is the first iteration:} \\ \quad S_d \leftarrow R_{(2i-1)\%4}\Big(R_{(2i-2)\%4}(S_d) \oplus S_{t_1} \oplus K_1 \oplus c^f_{2(i-1)}\Big) \\ \qquad \oplus S_{t_0} \oplus K_{(i-1)\%2} \oplus c^f_{2i-1}, \\ \quad S_{t_0} \leftarrow S_{t_0} \oplus K_{(i-1)\%2}, \\ \quad S_{t_1} \leftarrow S_{t_1} \oplus K_{i\%2} \\ \\ \text{otherwise:} \\ \quad S_d \leftarrow R_{(2i-1)\%4}\Big(R_{(2i-2)\%4}(S_d) \oplus S_{t_1} \oplus c^f_{2(i-1)}\Big) \\ \qquad \oplus R_{(i-1)\%4}(S_{t_0}) \oplus K_{(i-1)\%2} \oplus c^f_{2i-1}, \\ \quad S_{t_0} \leftarrow R_{(i-1)\%4}(S_{t_0}) \oplus K_{(i-1)\%2}, \\ \quad S_{t_1} \leftarrow R_{(i-1)\%4}(S_{t_1}) \oplus K_{i\%2} \end{cases}$$

where i denotes the number of iterations regarding R_{f_1}. The range of i is $1 \leq i \leq 5$ and $1 \leq i \leq 4$ for Dialga-256 and Dialga-256†, respectively. Therefore, R_f is defined as

$$R_f(P, T, K) = R_{f_1}(P \oplus T_0 \oplus K_0 \oplus K_1, T, K)^\alpha,$$

where α denotes the number of iterations in R_f depending on the claimed security, i.e., α will be 5 and 4 for Dialga-256 and Dialga-256†, respectively.

R_m R_m consists of two rounds of data processing and one tweak update. For S_{t_0}, the update involves one tweak update with one key addition. For S_{t_1}, the tweakey update involves one tweak update with SubCell and its inverse SubCell^{-1}. One iteration of R_m updates S_d, S_{t_0}, and S_{t_1} as follows:

$$R_{m_1}(S_d, S_t, K) = \begin{cases} S_d \leftarrow R_{(2\alpha+1)\%4}(R_{2\alpha\%4}(S_d) \oplus S_{t_1} \oplus \mathsf{SubCell}(S_{t_0}) \oplus c^m_0) \\ \qquad \oplus MS\Big(R^{-1}_{(\alpha-1)\%4}\big(S_{t_0} \oplus K_{(\alpha-1)\%2}\big)\Big) \oplus c^m_1, \\ S_{t_0} \leftarrow R^{-1}_{(\alpha-1)\%4}\big(S_{t_0} \oplus K_{(\alpha-1)\%2}\big), \\ S_{t_1} \leftarrow R^{-1}_{(\alpha-1)\%4}\big(S_{t_1} \oplus K_{\alpha\%2}\big) \end{cases}$$

R_b R_b has an iterated construction that one iteration consists of two rounds of data processing and one tweakey update for S_{t_0} and S_{t_1}. One iteration of R_b updates S_d, S_{t_0}, and S_{t_1} as follows:

$$R_{b_1}(S_d, S_t, K) = \begin{cases} \text{if } i \text{ is the last iteration:} \\ \quad S_d \leftarrow R_{(2(\alpha+i)+1)\%4}\Big(R_{2(\alpha+i)\%4}(S_d) \oplus MS(S_{t_1}) \oplus c^b_{2(i-1)}\Big) \\ \qquad \oplus MS\big(S_{t_0} \oplus K_{(\alpha-i-1)\%2}\big) \oplus c^b_{2i-1}, \\ \quad S_{t_0} \leftarrow S_{t_0} \oplus K_{(\alpha-i-1)\%2}, \\ \quad S_{t_1} \leftarrow S_{t_1} \oplus K_{(\alpha-i)\%2} \\ \\ \text{otherwise:} \\ \quad S_d \leftarrow R_{(2(\alpha+i)+1)\%4}\Big(R_{2(\alpha+i)\%4}(S_d) \oplus MS(S_{t_1}) \oplus c^b_{2(i-1)}\Big) \\ \qquad \oplus MS\Big(R^{-1}_{(\alpha-i-1)\%4}\big(S_{t_0} \oplus K_{(\alpha-i-1)\%2}\big)\Big) \oplus c^b_{2i-1}, \\ \quad S_{t_0} \leftarrow R^{-1}_{(\alpha-i-1)\%4}\big(S_{t_0} \oplus K_{(\alpha-i-1)\%2}\big), \\ \quad S_{t_1} \leftarrow R^{-1}_{(\alpha-i-1)\%4}\big(S_{t_1} \oplus K_{(\alpha-i)\%2}\big) \end{cases}$$

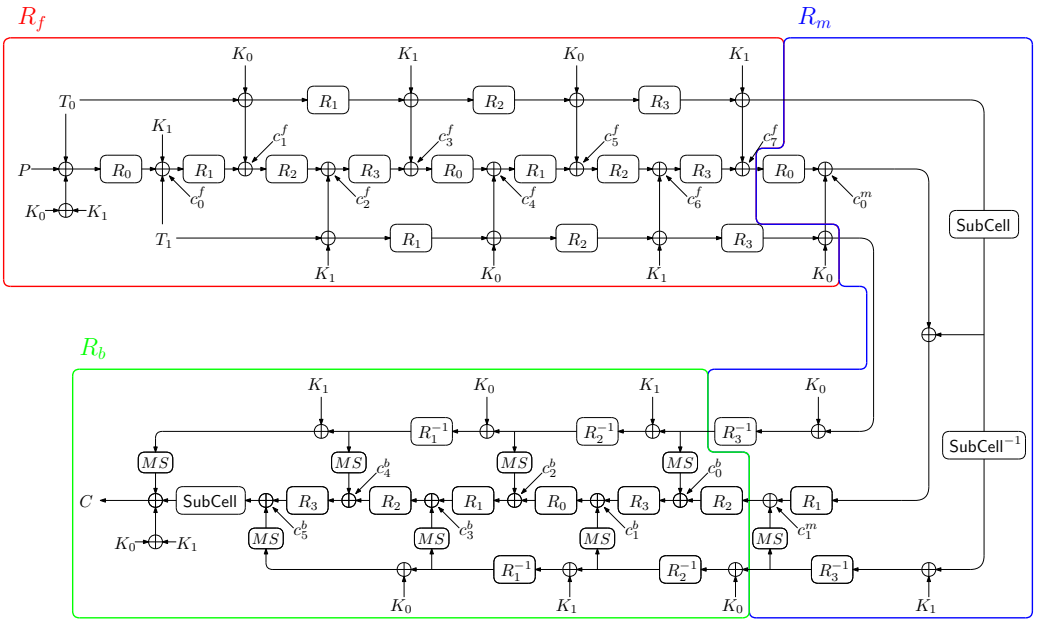

Figure 4: Overview of Dialga-256† (r_d=16).

where i denotes the number of iterations regarding R_{b_1}. In R_b, one SubCell is applied, and then $MS(S_{t_1})$ and $K_0 \oplus K_1$ are XORed with S_d to generate the ciphertext after 4 and 3 iterations of R_{b_1} for Dialga-256 and Dialga-256†, respectively. Therefore, R_b is defined as

$$R_b(S_d, S_t, K) = \mathsf{SubCell}(R_{b_1}(S_d, S_t, K)^\beta) \oplus MS(S_{t_1}) \oplus K_0 \oplus K_1,$$

where β denotes the number of iterations in R_b depending on the claimed security, i.e., β will be 4 and 3 for Dialga-256 and Dialga-256†, respectively.

2.3 Claimed Security

Dialga-128 and Dialga-256 claim 256-bit security in both the single-tweak setting and the related-tweak setting where available data is up to 2^{128}. They do not claim any security in the related-key and known/chosen-key settings. The number of rounds of Dialga-128 and Dialga-256 is 20.

We also propose the reduced-round variants named Dialga-128† and Dialga-256†, claiming 256-bit security when available data is limited to 2^{80}, which is the same setting of QARMAv2. Specifically, the number of rounds of Dialga-128† and Dialga-256† are both 16. Based on our analysis in Section 4, restricting data can weaken statistical attacks. For example, the number of rounds for the best differential distinguisher is reduced from 10 to 8.

Table 6 lists the parameter choices and security claims for the proposed variants, where ε is a small number, such as 16, used to absorb simple optimizations of the attacks, following the claims of QARMAv2 [ABD+23].

2.4 Shorter Key Variants

Depending on the application, a 256-bit key may not be necessary. In such cases, a shorter key, e.g., 128 and 192 bits, can be padded to a 256-bit length, allowing Dialga-128 and

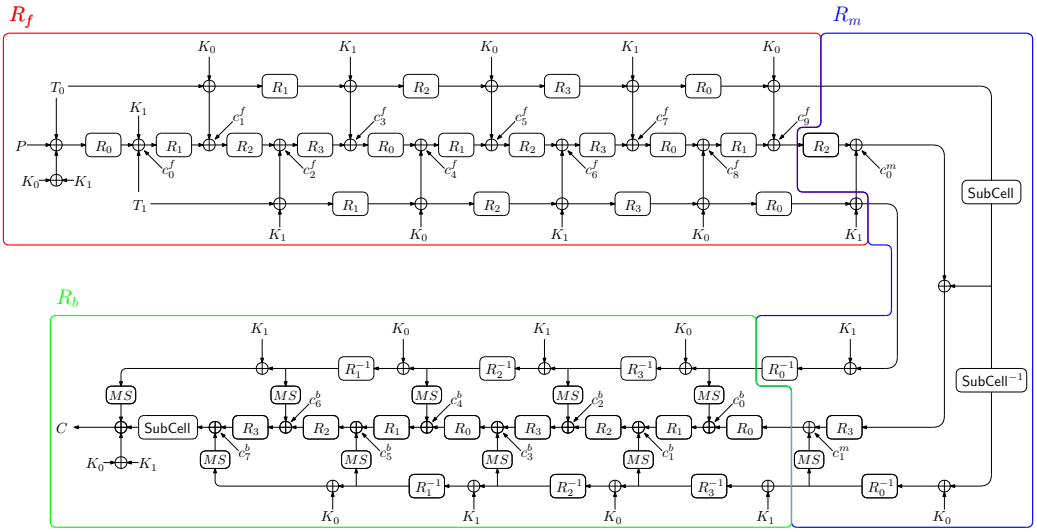

Figure 5: Overview of Dialga-256 (r_d=20).

Table 6: Parameter choices and security claims for the proposed variants, where ε is a small number, such as 16.

Variant	Block Size	Key Size	Tweak Size	# Rounds	Time	Data
Dialga-128†	128 bits	256 bits	128 bits	16	$2^{256-\varepsilon}$	2^{80}
Dialga-128	128 bits	256 bits	128 bits	20	$2^{256-\varepsilon}$	2^{128}
Dialga-256†	128 bits	256 bits	256 bits	16	$2^{256-\varepsilon}$	2^{80}
Dialga-256	128 bits	256 bits	256 bits	20	$2^{256-\varepsilon}$	2^{128}

Dialga-256 to be used as is. Since Dialga-128 and Dialga-256 are designed to ensure security for a 256-bit key, these variants provide an additional security margin. The claimed security of these variants depends on the key size, and data limitations remain the same as in the original version. As shown in Section 5, these variants still have an advantage over the 128- and 192-bit key variants of QARMAv2.

2.5 Version for Pointer Authentication and Memory Integrity

We also introduce 12-round versions of Dialga for use in pointer authentication and memory integrity, denoted by Dialga-128-PA and Dialga-256-PA, in line with the motivation behind QARMAv2 [ABD+23]. As illustrated in Figures 6 and 7, these variants retain the same specification as Dialga-128 and Dialga-256, respectively, except that the number of rounds is set to 12. When the target application allows only a single tweak block (e.g., a single "salt" in PAC), the second tweak block may be derived as $T_1 = \varphi(T_0)$ with $T_0 = T$, similarly to the approach of QARMAv2.

For memory-integrity applications, Dialga-PA may be used either in a mode that ensures temporal uniqueness (freshness) through counters or in a mode without such guarantees, depending on the underlying system architecture. To produce a pointer authentication code or a memory-integrity tag, the output is truncated to z bits, typically with $1 \leq z \leq 32$; then, available data is up to 2^z.

The main purpose of this truncation is to discourage potential attacks rather than to offer absolute cryptographic protection. As a result, the effective security is determined primarily by the tag length; for instance, with a 32-bit tag, the success probability of

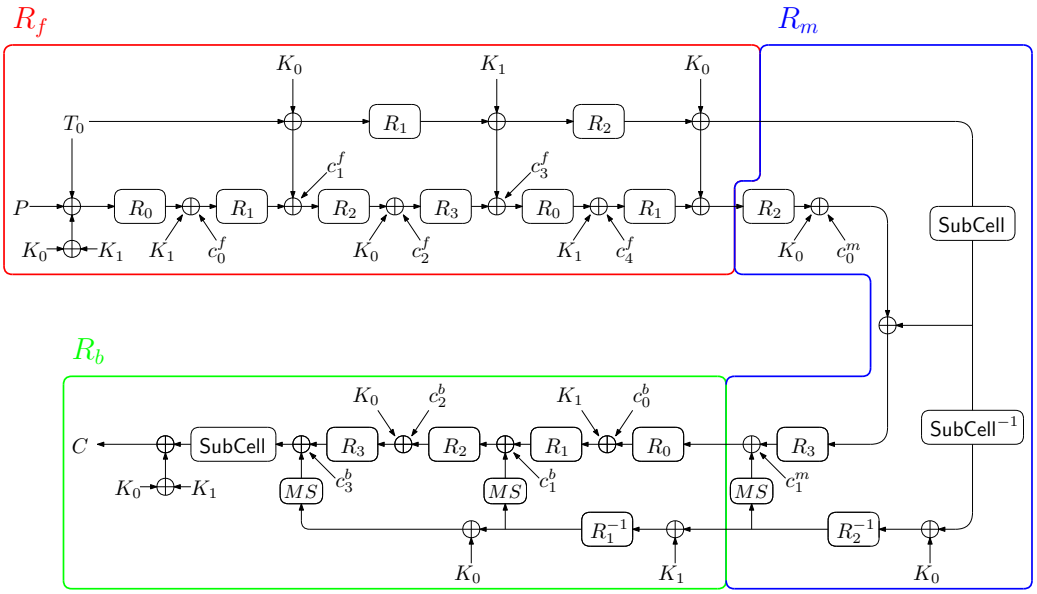

Figure 6: Overview of Dialga-128-PA.

a random forgery is at most about 2^{-32} per attempt. Moreover, because the output is heavily truncated, it becomes extremely difficult for an adversary to take advantage of differential characteristics, which substantially limits the applicability of more advanced cryptanalytic techniques. Any remaining cryptographic attack would also likely be confined to a known-plaintext setting, or to scenarios in which the adversary has only very limited control over the inputs in practice. Consequently, the attacker is essentially reduced to either blindly guessing the tag or performing an exhaustive key search, while the 12-round Dialga core remains a lightweight and secure underlying primitive.

3 Design Rationale

3.1 General Construction

Our primary objective is to design a TBC with a 256-bit key and a 256-bit tweak that provides competitive latency performance to Gleeok in the unrolled implementation. Additionally, we prioritize minimizing area to facilitate deployment in diverse devices and applications, including unified encryption/decryption circuits.

3.1.1 Midori-type Block Cipher

The data processing part of Dialga is based on the 128-bit block cipher Midori-128 [BBI+15], which employs an SPN (Substitution-Permutation Network) structure. The r-th round function consists of S-box S, cell permutation $\pi_r(x)$, matrix M and key addition operations with the round keys for the r-th round denoted as $(RK_0^r, \ldots, RK_{15}^r)$ as shown in Figure 8, where these operations are performed bitwise on m-bit data. Midori-type block cipher has a block length of $m \times 16$ bits, where for $m = 4$, it becomes a 64-bit block cipher, and for $m = 8$, a 128-bit block cipher.

While Midori-128 was originally designed for low-energy purposes, its components, particularly the S-box and matrix, are also highly suitable for low-latency applications due to their impressively small delay and low area. Furthermore, the involution property

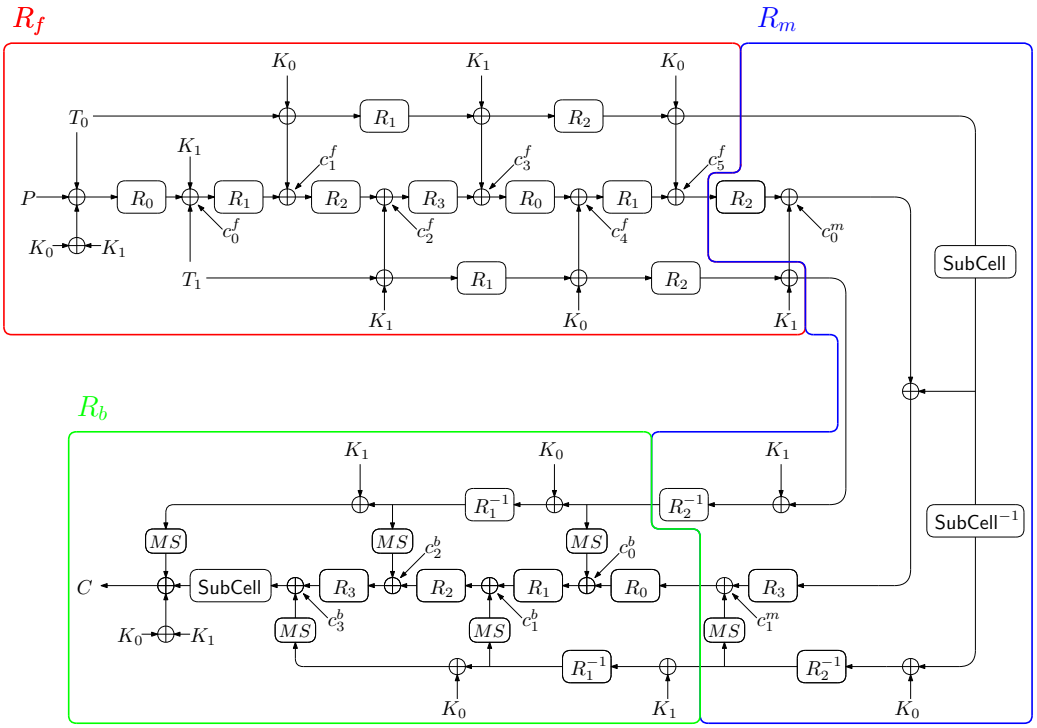

Figure 7: Overview of Dialga-256-PA.

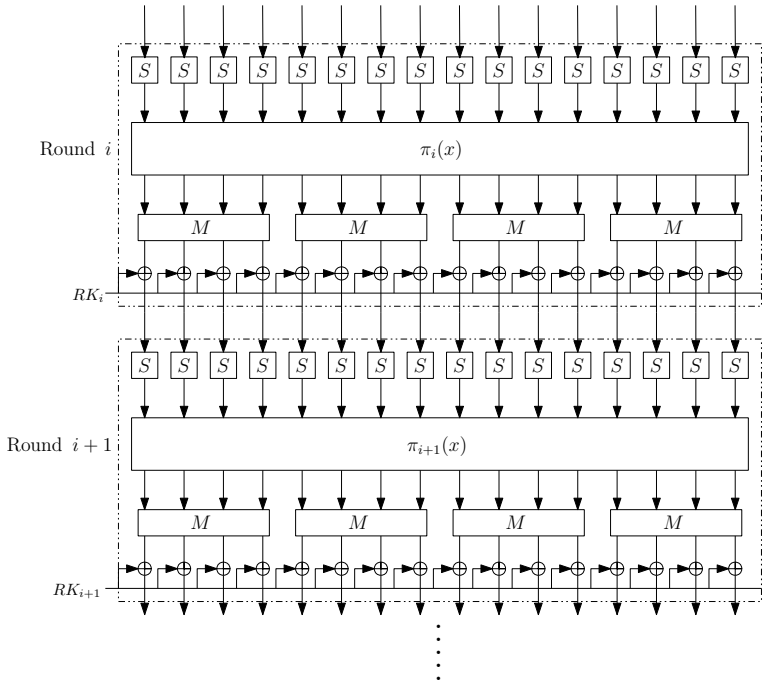

Figure 8: Midori-type block cipher using multiple cell permutations.

of the S-box and matrix enables the construction of unified encryption/decryption circuits with minimal overhead compared to encryption-only designs.

3.1.2 Design Space of Midori-type Cipher for Low-Latency Purpose

At ASIACRYPT 2015, Banik et al. demonstrated the existence of 1,152 equivalent classes of cell permutations for Midori [BBI+15], each offering identical lower bounds on the number of active S-boxes and diffusion properties. At FSE 2019, Alfarano et al. proved that these permutations are optimal among all classes of byte permutations of $2^{44.35}(= 16!)$ in terms of these metrics [ABI+18].

In the implementation of low-latency ciphers, an unrolled implementation, where all rounds are implemented in hardware rather than on a per-round basis, is a main target for real-time encryption. Consequently, there is no need to use the same function for each round. In addition, in hardware, cell permutations can be implemented without incurring additional costs, meaning that varying the permutations in each round does not introduce significant overhead.

Thus, the design space of using multiple linear layers has not been thoroughly explored. In this paper, multiple linear layers refer to the use of multiple cell permutations within a single cryptographic algorithm.

These layers have the potential to enhance the security of Midori-type scheme with negligible overhead in the unrolled implementation, leading to a reduction in overall latency. Specifically, our research question is as follows:

Can multiple linear layers in Midori-type block ciphers improve the security?

3.2 Introducing New Linear Layer: Multiple Cell Permutations

In this section, we examine a Midori-type block cipher that leverages multiple cell permutations, aiming to find better ones that significantly enhance security compared to the original Midori. In particular, we focus on the resistance to differential attacks as several low-latency ciphers have been broken by differential attacks [BDBN23, WBD+24, WNL+23]. We also consider three S-boxes with low latency and small area and then evaluate how the combined effect of the S-boxes and the linear layer influences the overall security against several attacks, where the matrix in the linear layer remains the binary involution matrix used in Midori for efficiency reasons.

3.2.1 Target Cell Permutations

When considering multiple cell permutations, the number of potential linear layer combinations grows exponentially with the number of rounds, rendering exhaustive exploration impractical. To address this, we focus on linear layers that utilize multiple cell permutations within fixed-length intervals. Specifically, we examine 2, 3, and 4-round intervals, each employing distinct permutations in Midori-type block cipher.

Since the cell permutation is defined over the set $\{0, 1, \ldots, \mathrm{f}\}$, there are approximately $16! \approx 1.0 \times 10^{13}$ possible permutations. Consequently, the number of candidates for linear layers over 2, 3, or 4 rounds is estimated as $2^{86.4}(\approx (1.0 \times 10^{13})^2)$, $2^{129.6}(\approx (1.0 \times 10^{13})^3)$, and $2^{172.7}(\approx (1.0 \times 10^{13})^4)$, respectively.

Due to the difficulty in exploring all candidates for these linear layers, we limit our scope to 1,152 equivalent classes of cell permutations for Midori [BBI+15]. It is because these classes have been thoroughly studied and are recognized as optimal for single permutations, particularly with respect to the lower bounds of active S-boxes and diffusion properties [BBI+15, ABI+18]. If we are able to identify more effective multiple-cell permutations than these permutations, this could serve as evidence that this approach is promising for achieving our objectives.

Table 7: Hardware characteristics and cryptanalytic properties of the S-boxes where $\mathcal{A}$: area-optimized, $\mathcal{L}$: delay-optimized. DP, LP, and FD indicate differential probability, linear probability, and full diffusion.

S-box	Area(GE)		Delay(ps)		DP	LP	FD	Involution
	$\mathcal{A}$	$\mathcal{L}$	$\mathcal{A}$	$\mathcal{L}$				
Sb_0 (Midori-64)	15.00	34.25	**7.42**	**4.81**	2^{-2}	2^{-2}	-	✓
Sb_1 (Midori-128)	16.25	30.50	9.85	7.37	2^{-2}	2^{-2}	✓	✓
Orthros	16.50	30.50	**8.85**	**5.18**	2^{-2}	2^{-2}	✓	-
Orthros (inv.)	21.90	66.50	19.65	7.11	2^{-2}	2^{-2}	✓	-
σ_0 (QARMA)	15.25	48.75	11.19	5.68	2^{-2}	2^{-2}	-	✓
PRINCE_0	16.75	30.25	14.00	6.05	2^{-2}	2^{-2}	✓	-

When considering linear layers of 2, 3, or 4 rounds using these 1,152 permutations, the number of candidates is stated as $2^{20.3}(\approx (1152)^2)$, $2^{30.5}(\approx (1152)^3)$, and $2^{40.7}(\approx (1152)^4)$, respectively.

While all combinations for a 2-round interval can be fully explored, exploring all combinations for 3 and 4-round intervals remains computationally infeasible. Therefore, we randomly select cell permutations from the 1152 candidates, using the pseudo-random number generator from the Python library "random", allowing duplicates in selecting 3 or 4 cell permutations. More precisely, 100 permutations are randomly chosen from the 1152 types for a 3-round interval, resulting in $2^{19.9}(\approx 100^3)$ combinations, and 30 permutations for a 4-round interval, resulting in $2^{19.6}(\approx 30^4)$ combinations.

3.2.2 Target S-boxes

We selected three S-boxes with low area and small delay to combine with the linear layers. Specifically, we considered Sb_0 and Sb_1 used in Midori-64 and Midori-128, respectively, and the S-box used in Orthros as candidates for the nonlinear layer. Table 7 shows the hardware characteristics and cryptanalytic properties of these S-boxes. While Sb_0 and Orthros's do not have full diffusion or involution property, their significantly small delay has great potential to improve the overall delay of the round function.

3.3 Search Method and Results

To identify the optimal combination of cell permutations and S-boxes, we employ a three-step approach. First, we perform a computationally efficient byte-wise evaluation of cell permutations, focusing on the security of differential attacks and diffusion properties while disregarding S-box specifics. This step enables rapid filtering of a large candidate pool. Second, we conduct a bit-wise evaluation, incorporating the differential distribution table (DDT) of the S-box to find the best candidates for S-box and permutation combinations. Finally, we examine the security of the remaining candidates against linear, impossible differential, and integral attacks to identify the best one. These evaluations are performed by state-of-the-art efficient evaluation methods by SAT solvers.

3.3.1 1st Step: Byte-wise Evaluation

We evaluate the lower bounds of the number of active S-boxes and byte-wise diffusion properties for variants employing target cell permutations in Section 3.2.1.

Active S-boxes. Since Midori requires 13 rounds to guarantee 64 active S-boxes, which is necessary for 128-bit security, our goal is to find combinations of cell shuffles that achieve

Table 8: Lower bounds of the number of active S-boxes of Midori and candidates employing multiple cell shuffles. Grey numbers indicate the worst values, while bold numbers indicate values better than those of Midori with respect to the number of active S-boxes.

Round		4	5	6	7	8	9	10	11	12	13	#comb.*1	#comb.*2	Total #comb.
2-round interval		-	-	-	-	-	-	-	-	≤63	-	-	–	$2^{20.3}$
3-round interval	best	16	23	30	34	**40**	**43**	**52**	**58**	**64**	-	-	44981	$2^{19.9}$
	Class 1	16	23	30	34	**40**	**43**	**52**	57	**64**	-	12322		
	Class 2	16	23	30	34	36	**42**	**52**	**58**	**64**	-	13963		
4-round interval	best	16	23	30	35	**40**	**45**	**52**	**59**	**64**	-	-	44083	$2^{19.6}$
	Class 3	16	20	28	34	**40**	**45**	**52**	**58**	**64**	-	690		
	Class 4	16	23	30	35	38	**42**	**52**	**59**	**64**	-	1546		
Midori [BBI+15]		16	23	30	35	38	41	50	57	62	67	-	–	–

*1 The number of combinations that achieve 64 active S-boxes over 12 rounds.
*2 The number of combinations where the lower bound of active S-boxes is best in 11, 10, 8, and 10 for Classes 1, 2, 3, and 4, respectively.

Table 9: Examples of cell permutations of Classes 1, 2, 3, and 4

x			0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
3-round interval	Class 1	$\pi_1(x)$	2	c	7	9	1	f	4	a	3	d	6	8	0	e	5	b
		$\pi_2(x)$	2	c	7	9	1	f	4	a	3	d	6	8	0	e	5	b
		$\pi_3(x)$	0	7	a	d	9	e	3	4	6	1	c	b	f	8	5	2
	Class 2	$\pi_1(x)$	2	c	7	9	1	f	4	a	3	d	6	8	0	e	5	b
		$\pi_2(x)$	2	c	7	9	1	f	4	a	3	d	6	8	0	e	5	b
		$\pi_3(x)$	6	0	f	9	5	3	c	a	7	1	e	8	4	2	d	b
4-round interval	Class 3	$\pi_1(x)$	8	f	6	1	b	c	5	2	a	d	4	3	9	e	7	0
		$\pi_2(x)$	3	8	d	6	1	a	f	4	0	b	e	5	2	9	c	7
		$\pi_3(x)$	3	8	d	6	1	a	f	4	0	b	e	5	2	9	c	7
		$\pi_4(x)$	8	f	6	1	b	c	5	2	a	d	4	3	9	e	7	0
	Class 4	$\pi_1(x)$	8	f	6	1	b	c	5	2	a	d	4	3	9	e	7	0
		$\pi_2(x)$	8	f	6	1	b	c	5	2	a	d	4	3	9	e	7	0
		$\pi_3(x)$	e	1	4	b	f	0	5	a	d	2	7	8	c	3	6	9
		$\pi_4(x)$	3	e	8	5	2	f	9	4	0	d	b	6	1	c	a	7

64 active S-boxes in fewer than 12 rounds. The search results for each class of combinations are shown in Table 8.

- 2-round interval combinations: There is no combination that attains 64 active S-boxes over 12 rounds.

- 3-round interval combinations: We find that 44,981 combinations achieve 64 active S-boxes over 12 rounds. Among these, the combinations with the highest lower bound of active S-boxes in each round are described as "best" in Table 8. As a result, no single combination maintains the "best" lower bound of active S-boxes across all rounds. However, there are 12,322 combinations (Class 1) where the lower bound of active S-boxes is "best" in 11 rounds and 13,963 combinations (Class 2) where it is "best" in 10 rounds. Examples of Classes 1 and 2 are given in Table 9.

- 4-round interval combinations: We find that 44,083 combinations achieve 64 active S-boxes over 12 rounds. Among these, there are 690 combinations (Class 3) where the lower bound of active S-boxes is "best" in 8 rounds and 1,546 combinations (Class 4) where it is "best" in 10 rounds. Notably, for Class 4, the lower bound of active S-boxes surpasses all rounds in Midori. Examples of Classes 3 and 4 are given in Table 9.

Table 10: Diffusion property of Classes 1, 2, 3, and 4

Diffusion(Forward/Backward Direction)		3/3	3/4	4/3	4/4	Total number of combinations
3-round interval	Class 1	7534	1098	1488	2202	12322
	Class 2	8414	1215	1786	2548	13963
4-round interval	Class 3	690	0	0	0	690
	Class 4	1546	0	0	0	1546
Midori [BBI+15]		1	-	-	-	-

Byte-wise Diffusion. In addition to the number of active S-boxes, we evaluate the byte-wise diffusion property in both forward and backward directions for combinations (Classes 1 to 4) that achieve 64 active S-boxes over 12 rounds. These results are presented in Table 10.

- 3-round interval: We find that 7,534 combinations in Class 1 and 8,414 combinations in Class 2 have full diffusion after 3/3 rounds in forward/backward directions.
- 4-round interval: We find that all 690 combinations in Class 3 and all 1,546 combinations in Class 4 have full diffusion after 3/3 rounds in forward/backward directions.

Results. Our results are particularly encouraging for Class 4 (1,546 combinations). Compared to the original Midori, Class 4 surpasses the lower bound of the number of active S-boxes in all rounds as shown in Table 8, while achieving a diffusion property equivalent to Midori. This suggests a potential improvement in security with fewer rounds.

3.3.2 2nd Step: Bit-wise Evaluation

In the next step, we focus on the 1546 candidates in Class 4 and identify the best combinations with three S-boxes in Section 3.2.2. The number of total combinations is estimated as $4,638$ $(= 1,546 \times 3)$.

Optimal Differential Characteristic Probability. For these combinations, we obtain the exact differential characteristic probability (DCP) through a bit-wise evaluation, incorporating the DDT of the S-box to find the optimal combination of S-box and cell permutations. The selection process is described as follows:

1. At the starting round of evaluation, we obtain optimal DCP among all candidates, identify candidates with the lowest DCP, and proceed to the next step with these candidates.
2. In the next round, we perform the same search for the remaining candidates as in Step 1 and repeat this process until only one candidate remains for each S-box.

In our evaluation, the number of starting rounds is five, which is the maximum number of rounds that allows for evaluating all 4,638 combinations. As shown in Table 11, the evaluation of up to 9 rounds enables us to narrow down to a single best combination for each S-box. These candidates are listed in Table 12, where variants with Sb_0, Sb_1, and Orthros S-box combined with multiple linear layers are referred to as Midori-S0, Midori-S1, and Midori-O, respectively.

Results. For all rounds, Midori-S0, Midori-S1, and Midori-O each provide significantly better security against differential attacks than the original Midori. Among them, Midori-S0 stands out as its DCP is lowest after 10 rounds.

Table 11: Lowest differential characteristics probability in each round, where () means the number of remaining candidate permutations.

r	1	2	3	4	5	6	7	8	9	10	11
Midori-S0	-	-	-	-	$\mathbf{51}_{(102)}$	$\mathbf{71}_{(9)}$	$\mathbf{81}_{(3)}$	$\mathbf{93}_{(1)}$	**107**	**126**	**128** $\leq$
Midori-S1	-	-	-	-	$\mathbf{50}_{(367)}$	$\mathbf{68}_{(121)}$	$\mathbf{80}_{(19)}$	$90_{(12)}$	$\mathbf{105}_{(1)}$	**123**	**128** $\leq$
Midori-O	-	-	-	-	$\mathbf{53}_{(298)}$	$\mathbf{73}_{(28)}$	$\mathbf{84}_{(22)}$	$\mathbf{95}_{(2)}$	$\mathbf{106}_{(1)}$	**124**	**128** $\leq$
Midori	2	8	14	32	49	67	79	90	96	114	132

Table 12: Best cell permutations for each S-box

x		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
Midori-S0	$\pi_1(x)$	7	0	d	a	5	2	f	8	4	3	e	9	6	1	c	b
	$\pi_2(x)$	d	0	a	7	b	6	c	1	2	f	5	8	4	9	3	e
	$\pi_3(x)$	7	d	a	0	6	c	b	1	5	f	8	2	4	e	9	3
	$\pi_4(x)$	d	8	6	3	e	b	5	0	c	9	7	2	f	a	4	1
Midori-S1	$\pi_1(x)$	d	0	a	7	b	6	c	1	2	f	5	8	4	9	3	e
	$\pi_2(x)$	7	d	a	0	6	c	b	1	5	f	8	2	4	e	9	3
	$\pi_3(x)$	d	0	a	7	b	6	c	1	2	f	5	8	4	9	3	e
	$\pi_4(x)$	7	9	2	c	1	f	4	a	8	6	d	3	e	0	b	5
Midori-O	$\pi_1(x)$	3	e	8	5	2	f	9	4	0	d	b	6	1	c	a	7
	$\pi_2(x)$	7	1	a	c	2	4	f	9	8	e	5	3	d	b	0	6
	$\pi_3(x)$	7	1	a	c	2	4	f	9	8	e	5	3	d	b	0	6
	$\pi_4(x)$	2	4	f	9	b	d	6	0	5	3	8	e	c	a	1	7

Table 13: Linear characteristics probability for Midori-S0, Midori-S1, and Midori-O

r	1	2	3	4	5	6	7	8	9	10	11	12
Midori-S0	2	8	14	32	**48**	**66**	**76**	**86**	**104**	**122**	**134**	-
Midori-S1	2	8	14	32	46	64	**76**	**86**	**100**	**118**	**128** $\leq$	-
Midori-O	2	8	14	32	46	62	74	**88**	**100**	**118**	**128** $\leq$	-
Midori [BBI+15]	2	8	14	32	46	64	74	84	92	110	124	136

3.3.3 3rd Step: Evaluation of Other Attacks

We examine the security of the remaining three candidates of Midori-S0, Midori-S1, and Midori-O against linear, impossible differential, and integral attacks to identify the best one.

Linear Attack. Table 13 shows linear characteristic probability (LCP) for Midori-S0, Midori-S1, and Midori-O. Midori-S0 and Midori-S1 provide better security against linear attacks than the original Midori for all rounds, and Midori-S0, Midori-S1 and Midori-O achieve an LCP of less than 2^{-128} within 11 rounds, while Midori requires 12 rounds.

Impossible differential and Integral Attacks. Table 14 shows the required number of rounds to ensure security against differential, linear, impossible, and integral attacks. Note that these estimations are based on distinguishers rather than key recovery. These details are given in Section 4. The number of rounds that ensure security against integral attacks for Midori-S0, Midori-S1, and Midori-O is 8 rounds, which is the same as Midori. For impossible differential attacks, the number of secure rounds is 6 for Midori-O and 7 for both Midori-S0 and Midori-S1, which is the same as Midori.

Table 14: The required number of rounds to ensure security against differential, linear, impossible, and integral attacks.

Attack	Differential	Linear	Integral	Impossible
Midori-S0	11	**11**	8	7
Midori-S1	11	**11**	8	7
Midori-O	11	**11**	8	**6**
Midori [BBI+15]	11	12	8	7

3.3.4 Decision

Table 15 shows the hardware characteristics of the round functions of Midori-S0, Midori-S1, Midori-O, and QARMAv2. Among these, Midori-S0 exhibits the smallest delay. According to our evaluation in Table 14, Midori-S0 achieves the security after 11 rounds as with Midori-S1 and Midori-O, while Midori requires 12 rounds. Moreover, Midori-S0 possesses the involution property, where the encryption and decryption functions are identical. For these reasons, we choose Midori-S0 as the underlying round function. Figure 14 illustrates round functions of Midori-S0.

Advantage. In the Midori-like cipher, the single permutation used in Midori is optimal from the perspective of active S-boxes (AS) [ABI+18]. Whether introducing multiple permutations could outperform this design has remained an open and non-obvious challenge. Our proposal is the first to reveal that multiple carefully-selected permutations can achieve a notable security enhancement over a (optimal) single permutation without additional overhead to the unrolled implementation for Midori-like cipher. This improvement enables a reduction in the number of rounds to achieve the required security. Another important advantage of introducing multiple linear layers is that it further expands the design space, increasing the number of viable permutation candidates. With a sufficiently large set of favorable permutations, and by carefully selecting S-boxes that align well with these permutations, the delay of the round function is significantly reduced, as shown in Table 15. These improvements collectively contribute to reducing overall latency.

Disadvantage. In round-based implementations, there is overhead in terms of area and energy, as a multiplexer for choosing different round functions is required. Detailed evaluation is shown in Appendix G. However, this lies outside the scope of our target, which focuses on low-latency applications using unrolled implementations.

Remarks. We would like to clarify that Midori-S1 is a variant that uses the same S-box as Midori-128 together with multiple permutations. As shown in Table 11, these results demonstrate that employing multiple permutations alone enhances the resistance of Midori-128 against differential attacks. Prior to these evaluations, we also analyzed a variant that retained the single linear layer of Midori-128 but replaced the S-box with Sb_0 as well as the S-boxes from Orthros and QARMA. However, we found that simply substituting the S-box does not significantly improve security. This indicates that the use of multiple permutations can indeed contribute to strengthening resistance against differential attacks.

Limitation. Our search focused on a small class of multiple cell permutations due to computational limitations, and it may be possible to find better permutations beyond this restricted scope. In addition, combining multiple permutations with multiple S-boxes remains a promising direction for further improvement. Nevertheless, in our evaluation, we

Table 15: Hardware characteristics of round functions where $\mathcal{A}$: area-optimised, $\mathcal{L}$: delay-optimised.

Target	Area (um2)		Delay (ps)	
	$\mathcal{A}$	$\mathcal{L}$	$\mathcal{A}$	$\mathcal{L}$
Midori-S0	231.9	410.0	38.26	**19.72**
Midori-S1	243.4	350.7	40.43	23.32
Midori-O	234.0	365.2	36.52	20.82
Midori-O (inv)	271.9	387.8	43.92	33.39
QARMAv2	247.4	338.1	41.69	32.12
Midori [BBI+15]	243.4	350.7	40.43	23.32

Table 16: Lower bounds of the number of active S-boxes in the related-tweak setting and the delay of a single round. The numbers in parentheses show an example of the individual number of active S-boxes in the data processing part ($\#AS_d$) and the tweakey scheduling part ($\#AS_t$ for Dialga-128 and Dialga-128†, $\#AS_{t_0}$ and $\#AS_{t_1}$ for Dialga-256 and Dialga-256†) of Dialga. The results show "$(\#AS_d/\#AS_t)$" for Dialga-128 and Dialga-128† and "$(\#AS_d/\#AS_{t_0}/\#AS_{t_1})$" for Dialga-256 and Dialga-256†.

Target	Data Limits	r											Delay (ps)
		4	5	6	7	8	9	10	11	12	13	14	
Midori	2^{128}	16	23	30	35	38	41	50	57	62	**67**	–	23.32
Dialga (single tweak)	2^{128}	16	23	30	35	38	43	52	59	**64**	–	–	
Dialga-128†	2^{80}	8 (4/4)	14 (10/4)	19 (4/15)	28 (13/15)	38 (38/0)	**42** (26/16)	52 (52/0)	59 (59/0)	64 (64/0)	– –	– –	
Dialga-128	2^{128}	8 (4/4)	14 (10/4)	19 (4/15)	28 (13/15)	38 (38/0)	42 (26/16)	51 (26/25)	58 (33/25)	**64** (64/0)	– –	– –	**19.72**
Dialga-256†	2^{80}	7 (6/0/1)	10 (4/0/6)	18 (7/7/4)	27 (13/7/7)	37 (21/0/16)	**42** (26/16/0)	50 (27/0/23)	51 (26/0/25)	54 (31/0/23)	59 (36/0/23)	68 (40/0/28)	
Dialga-256	2^{128}	7 (6/0/1)	10 (4/0/6)	18 (7/7/4)	27 (13/7/7)	37 (21/0/16)	42 (26/16/0)	50 (34/0/16)	58 (33/25/0)	**64** (64/0/0)	– –	– –	
QARMAv2-128 ($\mathscr{T}=1$)	2^{80}	–	–	6	–	26	–	**44**	–	67	–	–	32.12
QARMAv2-128 ($\mathscr{T}=2$)	2^{80}	–	–	5	–	16	–	32	–	**52**	–	61	

were unable to identify a better candidate than Dialga, largely because of the vast design space. These issues remain open problems.

3.4 Reflection Tweakey Scheduling using Multiple Linear Layers

In TBCs, ensuring security against differential attacks in the related-tweak setting is particularly challenging because adversaries can fully control the tweak values. To achieve a balance between strong security against such attacks and efficient implementation, we design a tweakey schedule function for Dialga that guarantees security against such attacks with a smaller number of rounds, especially in terms of key recovery. Furthermore, it is designed to minimize the overhead of decryption delay and hardware footprint.

Strong Security against Related-Tweak Differential Attacks. We utilize round functions that incorporate multiple cell shuffles, as employed in the data processing part, into the tweakey schedule functions. This approach ensures that once differences are introduced into the tweakey scheduling function, we can guarantee the DCP shown in Table 11, solely through the tweakey schedule function.

Table 16 shows the lower bounds of the number of active S-boxes in the related-tweak

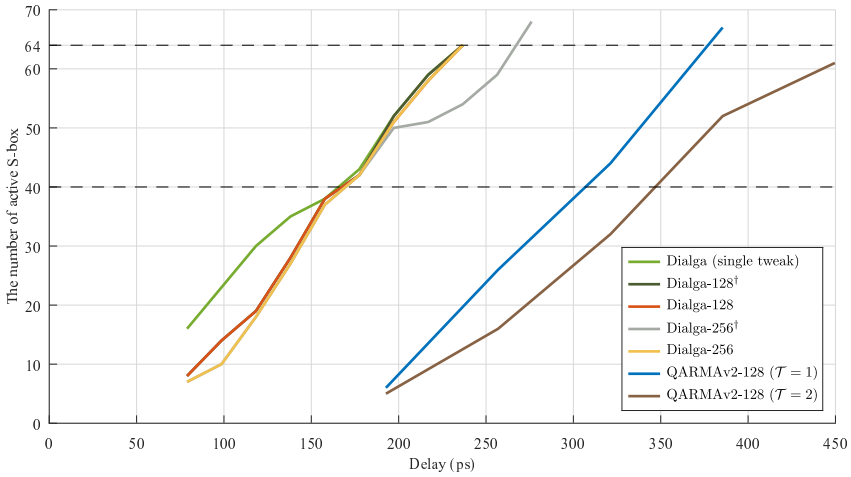

Figure 9: Relations between delay and the lower bounds of the number of active S-boxes in a related-tweak setting, where delay is estimated by the product of the delay of the single round function and the number of rounds.

setting, especially in terms of key recovery.[5] It can be seen from this table that the required security of Dialga-128 and Dialga-256 (i.e., the DCP of less than $2^{-2\times 64=-128}$) is achieved in 12 rounds, and the required security of Dialga-128† and Dialga-256† (i.e., the DCP of less than $2^{-2\times 40=-80}$) is achieved in 9 rounds. These results are equivalent to the required security in the single-tweakey setting. Moreover, although the gaps between the lower bounds of the number of active S-boxes for the QARMA family in the single-tweakey and related-tweakey settings are substantial (see Table 3 in [Ava17] for QARMA and Table 3 in [ABD+23] for QARMAv2), those for Dialga are marginal. These provide evidence that our tweakey function offers strong security against related-tweak differential attacks despite a smaller delay in the round function than Midori. Indeed, Dialga-128 and Dialga-256 achieve sufficient security with fewer rounds, even in a related-tweak setting, compared to Midori, which does not support a tweak.

Furthermore, we also address security against the boomerang attack, which is the most effective attack on QARMA. Our approach ensures a significant number of active S-boxes even in a small number of rounds, such as up to 6 rounds, resulting in strong security against this type of differential attack.

Finally, Figure 9 illustrates relations between delay and the lower bounds of the number of active S-boxes in a related-tweak setting. This figure corresponds to Table 16, and the delay is estimated by the product of the delay of the single round function and the number of rounds. For example, the delay of 10-round Dialga-256† is $19.72 \times 10 = 197.2$ ps (horizontal axis) and the minimum number of active S-boxes is 50 (vertical axis). It shows that Dialga achieves strong security against differential attacks in the related-tweak setting and maintains a small delay simultaneously, compared to QARMAv2. Specifically,

[5] When merely searching for a distinguisher, the initial round function in the tweakey schedule must be excluded, as it does not exist in our design. However, the primary goal of finding a good distinguisher is to efficiently recover the secret key. Appending several key recovery rounds in both the forward and backward directions is a natural extension of the process, and in this extended setting, it is essential to consider the initial round function of the tweakey schedule. Accordingly, Table 16 shows the lower bounds of the number of active S-boxes in the related-tweak setting, incorporating the initial round of the tweakey schedule. For reference, Appendix C shows the lower bounds of the number of active S-boxes in the related-tweak setting in the case of simply searching for a distinguisher.

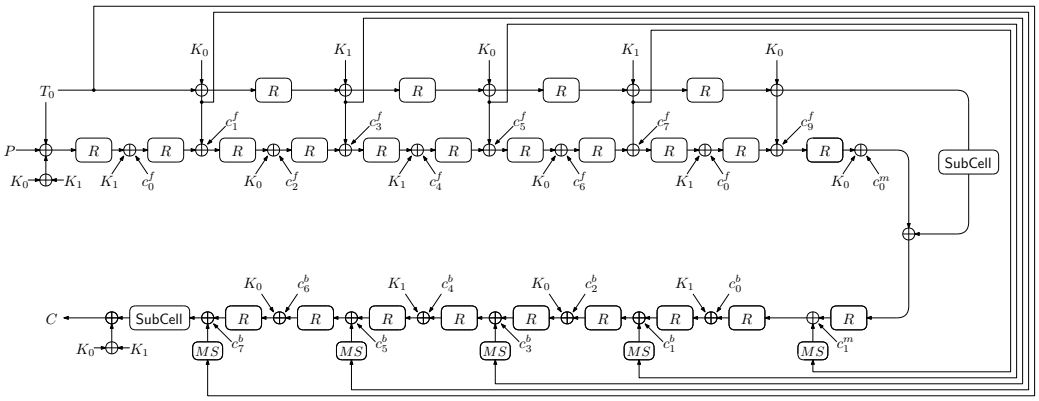

Figure 10: Another representation of Dialga-128.

Dialga achieves 64 active Sboxes within about 235 ps, demonstrating a near 50% reduction in delay compared to QARMAv2 with a 256-bit tweak.

Decryption w/o Delay Overhead and Minimizing Area. If we use the round functions in the tweakey schedule function on-the-fly, decryption requires generating round keys before starting the decryption process, leading to significant delay overhead. To address this issue, we utilize a reflection tweakey scheduling function. Specifically, we split it into two sections such that the function of the second part is the inverse of the function of the first part. With this function having the involution property, both encryption and decryption can be performed using the same tweakey scheduling function. To break the symmetry between the first and second parts, we utilize a byte shuffle operation in the second part before XORing the round keys with the data processing part.

Furthermore, the reflection property not only minimizes delay overhead but also reduces the area. By using an equivalent expression, the second part can be removed from the circuit. Note that, due to this fact, in our evaluation in Table 16, we do not count active S-boxes in the second part, as their differential characteristics are deterministic once the first part is determined. Figure 10 illustrates another representation of Dialga-128.

Remark. The difference in the number of rounds between the forward part (R_f) and the backward part (R_b) arises from the following two properties. First, in each of R_f, R_m, and R_b, a tweak schedule function is applied every two rounds. Second, the tweak schedule function satisfies a reflection property. Consequently, when a tweak schedule function is included in R_m, the number of rounds in R_f and R_b is slightly different.

4 Security Evaluation

We evaluated the security of Dialga against a wide range of cryptanalytic techniques, including differential, linear, impossible differential, boomerang, integral, invariant subspace, meet-in-the-middle, and reflection attacks. Our evaluation focuses on identifying the maximum number of rounds for which a distinguisher can be constructed, and on estimating the maximum number of rounds for which a key recovery attack remains feasible. As the best distinguisher is rarely compatible with the best key recovery strategy, we do not claim optimality in their combination.

4.1 Differential Attacks

The differential attack, proposed by Biham and Shamir [BS91], is one of the fundamental attacks used to ensure that the designed primitive has a sufficient level of security. We demonstrate here that Dialga is sufficiently resistant to differential attacks in the following three aspects: the lower bound of the number of active S-boxes, the maximum DCP, and the clustering effect. To efficiently estimate the maximum possible number of rounds for a successful key recovery attack, we assume that up to two rounds can be appended in both forward and backward directions because Dialga has full diffusion after 3/3 rounds in forward/backward directions, as discussed in Section 3.3.1. In addition, as claimed in Section 2.3, we need to consider that the data complexities available for the 16/20-round variants of Dialga are limited to $2^{80}/2^{128}$.

First, we focus on the lower bound of the number of active S-boxes. In both the single-tweak and related-tweak settings, it can be seen from Tables 8 and 16 that an attacker has the potential to construct differential distinguishers up to 8/11 rounds for the 16/20-round variants. From these results, since the attacker can append up to four key recovery rounds, we can estimate that a successful key recovery attack against the 16/20-round variants is limited to 12/15 rounds.

Next, we explore the maximum DCP. In the single-tweak setting, it can be seen from Table 11 that an attacker can construct differential distinguishers up to 5/10 rounds for the 16/20-round variants. We also evaluate the maximum DCP in the related-tweak setting, and our evaluation demonstrates that an attacker can construct differential distinguishers up to 7/9 rounds for the 16/20-round variants. As a result, we estimate a successful key recovery attack against the 16/20-round variants is limited to 9/14 rounds in the single-tweak setting and 11/13 rounds in the related-tweak setting.

Finally, we evaluate the clustering effect. Sakamoto, Ito, and Isobe [SII23] mentioned that a TBC with a linear tweak schedule (especially, QARMA) has good resistance to the clustering effect. However, they did not discuss such resistance for a TBC with a non-linear tweak schedule like Dialga. The clustering effect is one of the essential metrics used for an actual key recovery attack; thus, it is essential to evaluate whether Dialga has good resistance to the clustering effect or not.

We evaluate the clustering effect for Dialga under the following settings. 1) Our target is Dialga-256 because it has a higher degree of freedom for tweak differences than Dialga-128 and is suitable for exploring the impact of the clustering effect. 2) The overall structure after appending key recovery rounds has symmetry. In other words, there are exactly two more forward rounds than backward rounds. For example, when an attacker uses a differential distinguisher for 4/2/2 or 5/2/3 rounds in the forward/middle/backward directions, key recovery rounds can be appended up to 2 rounds in both the forward and backward directions. In this case, the first round function of the data processing part in the distinguisher is R_2. Conversely, when an attacker uses a differential distinguisher for 5/2/2 or 6/2/3 rounds in the forward/middle/backward directions, key recovery rounds can be appended up to 1 and 2 rounds in the forward and backward directions, respectively. In this case, the first round function of the data processing part in the distinguisher is R_1. At the same time, it should be noted that the starting round function of the tweak schedule part also varies in both example cases.

We employ a SAT-based automatic search method proposed by Sakamoto et al. [SII23] and obtain the following results:

- For the 8-round structure (specifically, 4/2/2 rounds in the forward/middle/backward directions), the maximum DCP is 2^{-49}, and its clustering effect is $2^{-43.679}$. Table 24 in Appendix D shows an example of input–output differences.

- For the 9-round structure (specifically, 5/2/2 rounds in the forward/middle/backward directions), the maximum DCP is higher than or equal to 2^{-91}, and its clustering

effect is $2^{-81.221}$. Table 25 in Appendix D shows an example of input–output differences.

- For the 10-round structure (specifically, 5/2/3 rounds in the forward/middle/backward directions), the maximum DCP is higher than or equal to 2^{-96}, and its clustering effect is $2^{-88.082}$. Table 26 in Appendix D shows an example of input–output differences.

- For the 11-round structure (specifically, 6/2/3 rounds in the forward/middle/backward directions), the maximum DCP is higher than or equal to 2^{-165}, and we could not find any differential path whose clustering effect is higher than 2^{-128}.

From these results, the attacker can use the 8/10-round distinguisher for a key recovery attack against Dialga-256†/Dialga-256; thus, we can estimate that a successful key recovery attack against Dialga-256†/Dialga-256 is limited to 12/14 rounds.

To summarize, the resistance of the 16/20-round variants to differential attacks has a security margin of at least 4/5 rounds, respectively. Therefore, we conclude that Dialga has good resistance to differential attacks.

Security of Pointer Authentication (PA) Mode. We further discuss the security of the PA mode based on the 12-round Dialga (Dialga-128-PA and Dialga-256-PA) against forgery attacks and key recovery attacks.

First, we consider forgery attacks on the PA mode. While the data limit of the PA mode is 2^{32}, the clustering effect for the 8-round structure is at most $2^{-43.679}$, as described above. Moreover, our evaluation of the clustering effect is not tailored to forgery attacks on the PA mode. If the evaluation were specialized for such attacks, the resulting probability would be even smaller. Therefore, we conclude that the PA mode based on the 12-round structure is secure against forgery attacks.

Next, we consider key recovery attacks on the PA mode. Since the tag is truncated to 32 bits or less, it becomes challenging to perform the filtering in the backward direction that is essential in key-recovery procedures. In addition, due to the data limit of the PA mode, it is impossible to construct a distinguisher for the 8-round structure. As mentioned earlier, under the full-diffusion assumption, at most two rounds can be appended in both forward and backward directions of the distinguisher. From these observations, we conclude that the PA mode based on the 12-round structure is secure against key recovery attacks.

4.2 Linear Attacks

The linear attack, proposed by Matsui [Mat93], is one of the most powerful attacks to symmetric-key primitives alongside the differential attack. To efficiently evaluate the longest linear distinguishers, we employ a SAT-based automatic search method proposed by Sun et al. [SWW18]. Dialga takes this attack into consideration in the process of designing the round function as described in Section 3.3.3. Specifically, in the single-tweak setting, the attacker cannot construct the linear distinguishers over 8 and 12 rounds with the data complexity of less than 2^{80} and 2^{128}, respectively. In the related-tweak setting, a linear mask in the data processing part is never canceled by a linear mask from the tweakey scheduling, i.e., the security in the related-tweak setting against this type of attack is reduced to that of the single-tweak setting. Considering the round function of Dialga achieves full diffusion after 3 rounds in both forward and backward directions, it is hard to append more than 6 rounds in the key recovery. Therefore, we expect that Dialga-128 and Dialga-256 can resist the linear attack.

4.3 Impossible Differential Attacks

In impossible differential attacks [BBS99], the attacker attempts to construct the impossible differences in which the input differences will never reach the output differences. As a SAT model to evaluate differential characteristics can explore the transition of the differences, we can evaluate the impossible differences by this model with a small modification. More specifically, we remove the objective function from the SAT model and fix the input and output differences. If the SAT model returns "UNSAT", the fixed input and output difference will never reach each other, i.e., an impossible difference. To efficiently estimate the longest impossible differences on Dialga, we put one active bit into both the input and output and evaluate their existence with a SAT-aided tool. For the longest impossible differences in the single-tweak setting, it has been discussed in Section 3.3.3. Specifically, the longest impossible differences can be found at 8 rounds.

In the related-tweak setting, we confirm that the impossible differences cannot be constructed over 8 rounds of the data processing in both Dialga-128 and Dialga-256 by any manners to add tweakey scheduling, which is identical to the case in the single-tweak setting, including the patterns of 4/2/2, 5/2/2, 5/2/3, and 6/2/3 rounds evaluated in Sect. 4.1. Considering that full diffusion can be achieved in 3 rounds in both forward and backward directions, and the data complexity that the attacker can exploit is limited up to 2^{80} for the 16-round variant and 2^{128} for the 20-round variant, it is hard to append the key-recovery rounds of more than 6 rounds. Therefore, we expect that Dialga-128 and Dialga-256 can resist the impossible differential attack.

4.4 Boomerang-type Attacks

The boomerang attack, introduced by Wagner [Wag99], is a variant of the differential attack. This technique divides a cipher, denoted as E, into two sections: E_1 and E_0. This division allows the attack to penetrate more rounds that are otherwise resistant to differential attacks. Initially, two independent differential trails for E_1 and E_0 are combined to form the boomerang trail. However, Murphy later identified several issues regarding the independence of these trails [Mur11]. To address these issues, frameworks were developed that incorporate an E_m layer between the two sections [BK09, DKS10, DKS14]. A notable advancement in this area is the introduction of the *boomerang connectivity table* (BCT), which combines E_1 and E_0 with a one-round E_m layer [CHP+18].

For the analysis on Dialga, we consider one round of E_m. A SAT-based tool is employed to find optimal boomerang distinguishers with one round in the middle, taking the BCT effect into account. We analyze Dialga-256 in the related-tweak setting, as it is more susceptible to tweak differences. For 12-round Dialga-256, we found no trail with probability better than 2^{-128}. For Dialga-256†, the best boomerang trail is found for 11 rounds with a probability of approximately 2^{-88} when the round distribution of $E_0/E_m/E_1$ is 5/1/5. For 12-round Dialga-256† (the 16-round variant), no trails were found with a probability higher than 2^{-80}.

We adopted the strategies from [DDV20, DEFN22] to construct boomerang trails consisting of multiple rounds of E_m in the middle. In this scenario, the best boomerang trail for Dialga-256† is found for 11 rounds with a probability of approximately 2^{-105} when the round distribution of $E_0/E_m/E_1$ is 5/2/4. In this setting as well, we found no trails for 12-round Dialga-256† with a probability higher than 2^{-128}. Hence, we believe that Dialga offers strong resistance against boomerang attacks.

Truncated Boomerang Attacks. Dialga has been analyzed against truncated boomerang attacks [BL23], which was introduced at Eurocrypt 2023 and is based on the notion of truncated differentials [Knu94]. In Fig. 11, a 3-round truncated differential trail of Dialga-128 with probability 2^{-16} in the related-tweakey setting is presented. By extending

the initial part of this truncated differential, it can be shown that at most a 5-round truncated boomerang trail can be constructed. For Dialga-256, a similar level of resistance against such attacks was observed. Therefore, it can be concluded that Dialga is secure against truncated boomerang attacks.

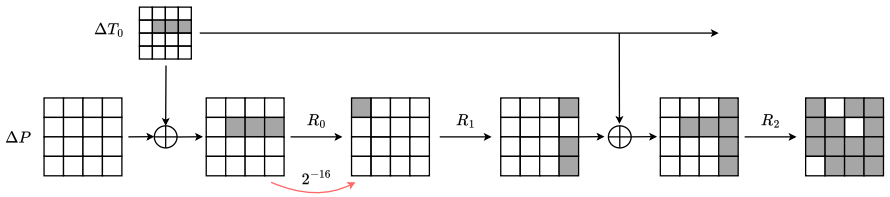

Figure 11: 3-round Truncated Differential Trail of Dialga-128

4.5 Integral Attacks

The most important part of the integral attacks is constructing the integral distinguisher so that the XOR of all elements in a set of input states becomes 0 at a certain output bit regardless of a key. A popular way to explore the integral distinguisher is to evaluate the division property, proposed by Todo [Tod15], by an MILP-based automatic search method proposed by Xiang et al. [XZBL16]. As their method can be converted into a SAT-based one, we evaluate the longest integral distinguishers by Xiang et al.'s method with SAT. The evaluation results in the single-tweak setting have been shown in Section 3.3.3. Specifically, we cannot find the integral distinguisher over 9 rounds.

In the related-tweak setting, for Dialga-128, the longest integral distinguisher never outperforms ones in the single-tweak setting with the time complexity of up to 2^{80} and 2^{128} regarding the number of rounds because the tweak is XORed to the plaintext before the data processing. For Dialga-256, we must consider that the attacker exploits all data in T_1. However, considering that the round function is applied to the tweaked scheduling and T_1 is XORed to the data processing after 1 round, it is hard to increase to 2 rounds; i.e., finding the 11-round integral distinguisher is challenging in any pattern of the number of forward, middle, and backward rounds. Since there are still enough margins to the full rounds of each variant, we expect that Dialga-128 and Dialga-256 can resist the integral attack.

4.6 Invariant Subspace Attacks

The invariant subspace attack, as introduced in [LAAZ11], is used to attack PRINTcipher [KLPR10]. This method entails identifying a subspace (a subset of all possible state and key values) that remains invariant by the round function. For iterated ciphers, this invariant characteristic can extend across the entire cipher (or multiple rounds) when certain weak keys are used. Consider a n-bit iterated cipher E with a round function $E_{k_i}(x) = R(x+k_i)$, where x is the n-bit state, k_i is the i-th round subkey and $R : \mathbb{F}_2^n \to \mathbb{F}_2^n$. Assume that there exists a subspace $U \subseteq \mathbb{F}_2^n$ and two constants $c, d \in \mathbb{F}_2^n$ such that

$$R(U + c) = U + d.$$

If a subkey $k_i \in u + c + d$ with $u \in U$, then:

$$E_{k_i}(U + d) = R(U + d + u + c + d) = R(U + c) = U + d.$$

Hence, if all the subkeys $k_i \in U + c + d$, then for all the plaintexts $P \in U + d$, the ciphertext $C \in U + d$ irrespective of the number of rounds.

In [BCLR17], this attack is formalized further, demonstrating that while the invariant property primarily exploits the similarity of round keys generated by a simple key schedule with minimal differences due to round constants, linear layers and the round constants play a crucial role in resisting such attacks. For Dialga, even though two keys, K_0 and K_1, are used alternately without a key scheduling function, tweaks produced by a non-linear schedule are added to the round keys along with the round constants which helps in resisting such attacks. Although the permutation layer varies across different rounds, the algorithm from [BCLR17, Appendix D] is run (assuming a single type of permutation layer) to establish that such attacks are successfully resisted. Furthermore, as mentioned in [GJN+16], due to the use of four different S-boxes SSb_0, SSb_1, SSb_2 and SSb_4 identical affine subspace transition is unlikely to occur which (as in the case of Midori-128) resists such attacks.

4.7 Meet-in-the-Middle Attacks

The 3-round full diffusion property enables us to ensure that any inserted key bit non-linearly influences all bits of the state after 3 rounds in both forward and backward directions. Consequently, the number of rounds used for partial matching is upper bounded by 5 (= (3-1) + (3-1) + 1). The initial structure (IS) condition requires that key differential trails in the forward direction and those in the backward direction do not share active non-linear components. Since any key differential affects all 16 S-boxes after at least 4 rounds in both forward and backward directions, no differential shares an active S-box for more than 4 rounds. Therefore, the number of rounds used for IS is upper bounded by 3. Assuming that the splice-and-cut technique allows an attacker to add 3 more rounds in the worst case, at most an 11-round (= 3+ 3+ 5) MitM attack may be feasible. However, due to the presence of white keys at the beginning and end, and the actual constraints of key orders, we consider it challenging to construct 11-round attacks on Dialga-128 and Dialga-256.

4.8 Reflection Attacks

Reflection attack [Kar08] exploits the self-similarity among the round function. This technique is further exploited to mount attack on PRINCE-like ciphers [SBY+13, SBY+15], described as $F_K^{-1} \circ M \circ F_K$, where F_K is a keyed permutation, F_K^{-1} is the inverse of F_K and M is a involution, i.e., $M = M^{-1}$. With reference to Section 2.2, the encryption procedure of both the variant of Dialga can be expressed as $R_b \circ R_m \circ R_f$. Note that, although R_m is not an involution, we can opt a modified representation to place the involution in the middle. Consider that the data processing and tweakey update are omitted from R_m to obtain an involution R'_m and the respective omitted operations are added to R_f and R_b to obtain R'_f and R'_b, respectively. Then the new encryption function can be represented as $R'_b \circ R'_m \circ R'_f$. However, $R'_b \neq (R'_f)^{-1}$, specifically due to the use of round constants and the application of a permutation before adding the tweaks in R'_b. Thus, we believe reflection attacks can not be applied on Dialga-128 and Dialga-256.

4.9 Comparison of Security Margins with Similar Designs

Since Dialga is designed as a low-latency block cipher building upon Midori, it is natural to compare the two. While Midori achieves efficiency through a single optimal cell permutation, Dialga strengthens security by employing multiple linear layers, optimal S-boxes, and a reflection-based tweakey schedule, while further reducing latency. Thus, comparison with Midori-128 highlights its improved security margin in terms of rounds. QARMAv2, as the state-of-the-art low-latency TBC with 256-bit tweak and key, serves as the main benchmark for Dialga. Dialga achieves comparable 256-bit security while offering significantly lower

Table 17: Comparison of security margins (in terms of the number of attacked rounds) between Dialga-256, Midori-128, and QARMAv2-128-256 with respect to differential, linear, integral, boomerang, and impossible differential attacks.

Cipher	#rounds	Maximum Attacked #rounds					Reference
		Differential	Linear	Integral	Boomerang	Impossible Differential	
Dialga-256	20	12	12	9	11	8	Section 4
Midori-128	20	12	12	7	14	7	[BBI+15, Section 5]
QARMAv2-128-256	32	20	16	–	22	10	[ABD+23, Table 2]

latency and competitive area efficiency. Hence, comparison with QARMAv2 demonstrates the advantages of Dialga at the highest security level and motivates round-based security margin analysis. In what follows, we compare the security margins of Midori, QARMAv2, and Dialga against standard cryptanalytic techniques, namely differential, linear, boomerang, and integral attacks. Among the variants of QARMAv2-128, which range from 24 to 32 rounds depending on the key size, we consider QARMAv2-128-256 (32 rounds) for comparison with Dialga-256 (20 rounds). Midori-128 also has 20 rounds in total.

For 13-round Midori-128, there were no differential or linear trails with probability greater than 2^{-128}. For both Dialga-128 and Dialga-256, these bounds are achieved in 12 rounds in the single-tweak as well as related-tweak settings. For QARMAv2-128-256, the maximum number of rounds reachable via differential and linear cryptanalysis is 20 and 16 (for finding a distinguisher), respectively. Using boomerang attack, the maximum attacked rounds for Dialga-256 is 11, whereas using the similar attack the maximum number of reachable rounds for Midori-128 and QARMAv2-128-256 is 14 and 22, respectively. For both versions of Dialga, the maximum number of rounds reachable via impossible differential is 8, whereas for Midori-128 and QARMAv2-128-256, the number is shown to be 7 and 10, respectively. For both variants of Dialga, the maximum number of rounds reachable via integral attack is 9, whereas for Midori-128 it is claimed that the maximum round can be 7 rounds. For QARMAv2-128-256, no details regarding integral attack is provided. The comparison of the security margins against these attacks are tabulated in Table 17.

Remarks on the Number of Rounds. The only difference between Dialga-128 and Dialga-256 lies in the key size; the data processing and tweak schedule functions remain identical. Therefore, in our evaluation, the number of rounds for the distinguisher is the same. Accordingly, the number of rounds was determined by considering the security margin for the key recovery case of Dialga-256. Although it might be possible to reduce the number of rounds for Dialga-128, since Dialga-256 allows greater computational effort in a key recovery attack than Dialga-128, we chose to keep the same number of rounds for both, prioritizing parameter simplicity. As a result, Dialga-128 has a larger security margin than Dialga-256, while its delay is smaller than that of QARMAv2-128.

5 Hardware Evaluation

The fully unrolled circuit of a block cipher is a huge combinatorial circuit that takes as input the plaintext and key/tweak and produces the ciphertext. If the target metric to be optimized is latency, circuit designers usually attempt to minimize the maximum signal delay between the input and output ports of the circuit, also commonly referred to as *critical path*. The critical path determines the smallest clock period (hence maximum clock frequency) the circuit can be theoretically clocked at. Hence, our primary focus was the fully unrolled circuit for Dialga.

5.1 Methodology

To do a fair evaluation, we follow the evaluation framework outlined in [BIL+21]. We construct our circuits using the `Nangate 15nm Open Cell Library` [MMR+15] since it is open source, and the results are easily reproduced. We adhered to the following design flow for evaluating each circuit.

1. The ciphers have been implemented in VHDL, and a functional simulation was done using the `Modelsim` software. The correctness of the constructed circuit is verified by checking if the circuit produces the desired output to some standard input test vectors.

2. We synthesize the circuits with *Synopsys Design Compiler*.

3. Optimizing the critical path is done by instructing the compiler to output a circuit with a very low delay value (say 1 `ps`) between all input/output ports. The compiler, of course, fails to do so but, in the process, outputs a circuit that is nearly optimized with a critical path. We denote the netlist so produced by the compiler as the delay-optimized circuit.

4. Additionally, we optimize the total area of the circuit. Optimizing for the area is done by using the compiler flag `compile_ultra`, which is a computationally heavy stage that outputs a circuit with, usually, near the minimal area. We denote this netlist as the area-optimized circuit.

5. Timing simulations are then performed on the synthesized netlist with a clock frequency fixed at 10 MHz. As a result of this process, the switching activity of each node of the circuit can be collected. This switching activity information can be used by the compiler to estimate the average power consumption of the circuit.

5.2 Results

We compare our design with the following ciphers, some of which are encryption standards and others which have recently appeared in literature and have low latency in hardware: `AES-{128|192|256}` [AES01], `MANTIS-{7|8}` [BJK+16], `QARMAv1-{64/128}` [Ava17], `QARMAv2` family [ABD+23], `PRINCE` [BCG+12], `PRINCE` v2 [BEK+20], Orthros [BIL+21], `SPEEDY-{6/7}` [LMMR21], `MIDORI-{64/128}` [BBI+15], `PRESENT-{80/128}` [BKL+07], `SKINNY-{64/128}` [BJK+16], Gleeok [ABC+24], `BipBip` [BDD+23a], `Aradi` [GMW24], along with `Scarf` [CGL+23], `Sonic-{256|512}/SuperSonic-{256|512}` [BDD+23b], and the recently proposed `Twinkle-256` [WHWL24]. We also consider the `ASCON-p`^{12} permutation used in a single key Even-Mansour mode. All results are presented in Table 18. As per the discussion presented in [ABD+23], we include a *Figure of Merit* (FoM), namely the product of Delay and Power divided by block size, normalized as a percentage of the `AES-128` value.

Among all block ciphers that offer at least 256 bits of security, i.e., `AES-256` and `QARMAv2-128-256`, both Dialga-256 and Dialga-128 (both 16 and 20 round versions) compare favorably in terms of both area, power consumption and total critical path. Specifically, Dialga-256† exhibit nearly half the delay of `QARMAv2-128-256` while achieving approximately a 40% reduction in area in both area and latency-optimized circuits, with the same claimed security. For area-optimized designs, a unified circuit can be realized with less than a 30% increase in area, while the delay is maintained in latency-optimized designs although with the cost of doubling circuit area. Dialga-128 supports 128-bit tweaks, so we may compare it with `QARMAv2-128-192` ($r = 13$), since the version of `QARMAv2-128-256` that supports 128-bit tweak also uses the 13-round core of `QARMAv2`$_r$ (as per Table 1b of [ABD+23]). The comparison in terms of both area and latency is favorable. Among the more recent designs

Algorithm Flat-ΘCB.$\mathcal{E}[\widetilde{E}_K](N, M)$

1 $(M[1], \ldots, M[m]) \xleftarrow{n} M$
2 $T \leftarrow \widetilde{E}_K^{N,0,0}(0^n)$
3 **for** $i = 1$ **to** $m - 1$
4 $\quad C[i] \leftarrow \widetilde{E}_K^{N,i,0}(M[i])$
5 $\quad T \leftarrow T \oplus \mathtt{msb}_\tau(M[i])$
6 $C[m] \leftarrow \widetilde{E}_K^{N,m,1}(M[m])$
7 $T \leftarrow T \oplus \mathtt{msb}_\tau(M[m])$
8 $C \leftarrow C[1] \| \ldots \| C[m]$
9 **return** (C, T)

Algorithm Flat-ΘCB.$\mathcal{D}[\widetilde{E}_K](N, C, T)$

1 $(C[1], \ldots, C[m]) \xleftarrow{n} C$
2 $T' \leftarrow \widetilde{E}_K^{N,0,0}(0^n)$
3 **for** $i = 1$ **to** $m - 1$
4 $\quad M[i] \leftarrow \widetilde{D}_K^{N,i,0}(C[i])$
5 $\quad T' \leftarrow T' \oplus \mathtt{msb}_\tau(M[i])$
6 $M[m] \leftarrow \widetilde{D}_K^{N,m,1}(C[m])$
7 $T' \leftarrow T' \oplus \mathtt{msb}_\tau(M[i])$
8 **if** $T = T'$ **then**
9 $\quad$ **return** $M \leftarrow M[1] \| \ldots \| M[m]$
10 **else return** $\perp$

Figure 12: The algorithms of Flat-ΘCB. (Left) Encryption, (Right) Decryption.

proposed in literature, i.e., `Aradi, Scarf, Sonic/Supersonic family, Twinkle`: Table 18 shows that they have decent performances as far as end-to-end latency is concerned. However, only `Scarf` supports a tweak in the design, and the full version of it may already have been cryptanalyzed [FLL+25].

As mentioned in Section 1.3, we report a number of additional implementation results in the appendix. Appendix E shows additional synthesis results using the UMC 40nm standard cell library. We also discuss circuits with unified Encryption/Decryption Paths in Appendix F, and implement Flat-ΘCB [IMO+22], a low-latency authenticated encryption mode suitable for memory encryption, using Dialga as a core component. In Appendix G, we discuss round-based and 2/4 round unrolled circuits. Finally, we discuss masked circuits in Appendix H.

As mentioned in Section 2.4, Dialga-128 and Dialga-256 can support shorter keys, such as 128- and 192-bit keys, by applying appropriate padding while maintaining the same number of rounds. Even in this scenario, Dialga-128 and Dialga-256 outperform `QARMAv2-128-128` and `QARMAv2-128-192` despite these shorter-key variants having a more security margin than Dialga-128 and Dialga-256 with a 256-bit key.

6 Low-latency Authenticated Encryption Mode

We implement Dialga as a core of a low-latency AE called Flat-ΘCB. It is developed as an AE component of a tree-based memory encryption scheme, ELM, introduced by Inoue et al. [IMO+22]. ELM is based on a TBC and has a similar structure to the TBC-based counterpart of seminal OCB mode, ΘCB. The advantage of Flat-ΘCB over ΘCB is that it has minimal latency in a fully parallel configuration, namely one TBC call, for both encryption and decryption for any tag length. Note that ΘCB needs two calls for decryption in general. Since decryption latency is critical for memory encryption, Flat-ΘCB is suitable for memory encryption. Figure 12 shows the pseudocode of Flat-ΘCB, where N, M, C, T denotes a nonce, a plaintext, a ciphertext, and a tag, respectively. $\widetilde{E}_K^{N,i,j}(X)$ denotes encryption of the TBC taking key K, message block $X \in \{0,1\}^n$ for a certain fixed n, and tweak $(N, i, j) \in \{0,1\}^\nu \times \{0,1\}^\ell \times \{0,1\}$. Here, ν denotes the nonce length, and ℓ denotes the counter length. The last component is used for domain separation. The decryption is denoted as $\widetilde{D}_K^{N,i,j}(X)$ in a similar manner. The Flat-ΘCB mode has n-bit security, or more precisely, the privacy (confidentiality) bound is just the computational security

Table 18: Comparative Evaluation Metrics for the Nangate 15nm Process and Library.

		Area optimized					*Latency optimized*				
		Area		Delay	Power	FOM	Area		Delay	Power	FOM
Cipher	Rounds	μm^2	GE	ps	mW	% AES	μm^2	GE	ps	mW	% AES
AES-128	10	22478	114329	1728	26.63	100	28071	142777	1026	31.14	100
AES-192	12	26698	135796	2061	34.96	157	34025	173063	1221	36.39	139
AES-256	14	31744	161459	2319	47.64	240	41601	211594	1422	49.12	219
PRESENT-80	31	3808	19370	1001	2.87	11.2	6261	31846	711	4.63	20.2
PRESENT-128	31	3953	20105	960	2.67	12.5	6446	32787	706	4.56	20.6
MIDORI-64	16	1929	9814	629	0.96	2.61	2567	13054	455	1.31	3.76
MIDORI-128	20	5102	25952	851	3.35	6.20	6976	35484	604	4.47	8.44
PRINCE	12	1664	8465	536	0.67	1.56	2338	11891	372	0.89	2.08
PRINCE v2	12	1775	9026	691	0.92	2.76	2589	13168	379	0.94	2.23
Orthros	12	5766	29329	486	2.33	2.46	7439	37838	352	2.64	2.90
Gleeok-128	12	9217	46879	488	1.82	1.93	12344	62784	359	2.52	2.83
	10	7645	38885	409	1.28	1.13	9757	49629	298	1.44	1.34
Gleeok-256	16	24476	124493	684	6.70	4.98	34152	173704	491	8.59	6.60
	12	18332	93241	514	3.76	2.10	24643	125342	438	4.52	3.10
SPEEDY-5	5	4624	23518	445	1.33	0.86	6235	31712	293	1.54	0.94
SPEEDY-6	6	5620	28584	545	2.06	1.63	7539	38346	355	2.37	1.76
SPEEDY-7	7	6610	33618	659	3.18	3.02	8875	45139	419	3.51	3.06
ASCON-p12	12	8562	43545	568	2.07	0.65	10408	52939	436	1.82	0.99
SKINNY-64-192	32	4069	20693	1953	5.03	42.6	6162	31332	892	5.30	29.6
SKINNY-128-128	40	18520	94200	3862	40.89	343	29643	150777	1763	49.47	196
SKINNY-128-384	40	20480	104172	4098	45.18	402	30896	157148	1751	47.32	186
MANTIS-7	16	2412	12268	723	1.23	3.87	3364	17110	474	1.32	3.92
MANTIS-8	18	2703	13749	814	1.56	5.52	3759	19121	529	1.66	5.50
BIPBIP-Dec	11	5344	27180	710	2.01	16.5	6554	33843	327	1.42	7.75
BIPBIP-Enc	11	7031	35760	1520	3.99	70.3	10334	52561	683	3.86	44.0
Aradi	16	5497	27960	1251	3.57	9.72	7536	38330	707	3.51	7.78
Scarf	8	1114	5666	355	0.18	1.76	1430	7271	219	0.20	1.72
Sonic-256	24	6304	32064	482	1.01	0.53	8007	40726	416	1.43	0.93
Sonic-512	24	12594	64056	482	2.05	0.54	15673	79718	418	2.90	0.95
SuperSonic-256	21	5975	30389	665	1.77	1.28	8400	42723	394	1.48	0.91
SuperSonic-512	21	11950	60779	660	3.51	1.26	15241	77520	405	2.94	0.93
Twinkle-256	$18\frac{1}{2}$	51899	263974	752	18.99	3.45	71890	365649	645	23.17	5.20
QARMAv2-128-128 ($r = 9$)	20	6336	32226	909	4.39	8.67	9292	47261	623	4.96	9.67
QARMAv2-128-128 ($r = 11$)	24	7585	38580	1140	6.24	15.5	11142	56674	749	7.19	16.9
QARMAv2-128-192 ($r = 13$)	28	8842	44975	1330	8.47	24.5	12991	66074	877	10.09	27.7
QARMAv2-128-256 ($r = 15$)	32	10107	51411	1516	11.08	36.5	14734	74941	1002	13.00	40.8
Dialga-256 (Enc only)	20	**7230**	36776	**883**	3.70	7.10	**9691**	49294	**626**	4.58	8.99
Dialga-256† (Enc only)	16	**5817**	29587	**747**	2.58	4.19	**7955**	40459	**507**	2.95	4.68
Dialga-256-PAC (Enc only)	12	**4378**	22266	**573**	1.52	1.89	**6054**	30791	**386**	1.67	2.03
Dialga-256 (Enc+Dec)	20	10302	52398	1470	7.62	24.3	18564	94424	662	9.19	19.1
Dialga-256† (Enc+Dec)	16	8302	42227	1219	5.01	13.28	14979	76189	535	5.64	9.44
Dialga-128 (Enc only)	20	6269	31883	905	3.86	7.60	8663	44062	622	4.55	8.86
Dialga-128† (Enc only)	16	5074	25807	717	2.34	3.65	7033	35770	502	2.92	4.59
Dialga-128-PAC (Enc only)	12	3896	19815	567	1.49	1.84	5398	27454	383	1.65	1.98
Dialga-128 (Enc+Dec)	20	9358	47597	1490	7.88	25.5	16806	85481	650	8.80	17.9
Dialga-128† (Enc+Dec)	16	7572	38513	1227	5.14	13.70	13619	69272	531	5.68	9.45

(indistinguishability from truly random tweakable permutation) of $\widetilde{E}$, and the authenticity bound is $2q_v/2^\tau$ for τ-bit tag and q_v decryption queries plus the computational security of $\widetilde{E}$ [IMO+22, Theorem 2]. To optimize performance as memory encryption (in particular those used in TEE), the scheme assumes there is no associated data (AD), or if it exists, it is absorbed in the tweak. We also assume integral message blocks, i.e., the last block is full n bits. To support arbitrary-length AD and partial last block, some modifications will be needed, namely PMAC-like AD processing and padding. The original paper did not specify them. However, this is possible without harming the provable security and latency, although the bandwidth will increase due to the padding.

A tweak (N, i, j) is encoded into Dialga's tweak space, and any injective encoding could be safely used. For our implementation, we adopt the following. For Flat-ΘCB using Dialga-256, the 256-bit tweak is a concatenation of the 128-bit nonce and 64-bit counter followed by unused 7 bytes and 1 byte for domain separation. For Flat-ΘCB using Dialga-128, the 128-bit tweak is a concatenation of the 64-bit nonce and 56-bit counter followed by 1 byte

Table 19: Performance of the Flat-ΘCB mode and AES-256-GCM using the Nangate 15 nm standard cell library.

		Area optimized					*Latency optimized*				
		Area		Delay	Power	Max TP	Area		Delay	Power	Max TP
Cipher	Rounds	μm^2	GE	ps	mW	Tbps	μm^2	GE	ps	mW	Tbps
AES-256-GCM	14	22724	115580	2860	15.87	0.041	26242	133474	2393	19.26	0.049
Dialga-256	20	15024	76417	1669	9.83	0.070	18827	95761	836	7.00	0.139
	16	12329	62707	1381	6.12	0.084	15447	78567	685	4.30	0.170
Dialga-128	20	13537	68854	1667	9.77	0.070	17613	89584	837	6.73	0.139
	16	11098	56445	1380	6.17	0.084	14419	73341	682	4.30	0.171

for domain separation. Note that 128-bit nonce allows random nonce to maintain 64-bit security, whereas if the nonce is totally deterministic (as in the case of Intel SGX or other memory encryption schemes in Tree-based TEEs), 64-bit nonce would be sufficient and more convenient in practice, so which version is suitable depends on the application. For counters we use an LFSR instead of a conventional up-counter for its hardware efficiency. The use of LFSR as a counter in tweak is already adopted by Romulus [GIK+19], a finalist of NIST lightweight cryptography project [nis17].

6.1 Evaluation in Hardware

We implement the circuit for Flat-ΘCB as shown in Figure 13. Since we need access to both the encryption and decryption functions of the underlying block cipher, we can use either the area or delay-optimized circuit of the combined block cipher as discussed in Appendix F. The circuit is self-explanatory: the block cipher input is either the all zero

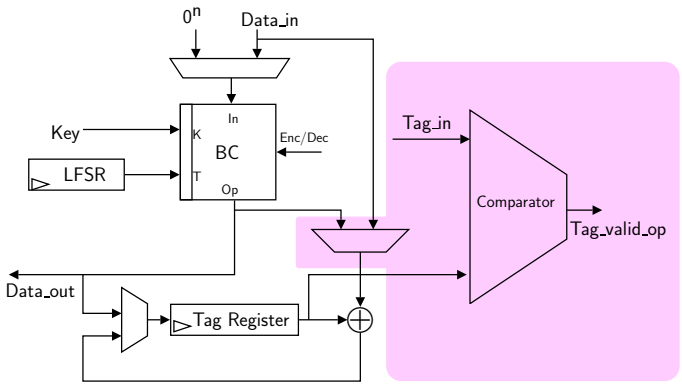

Figure 13: Implementation of Flat-ΘCB in hardware. The circuit components on the purple background are additionally needed for decryption.

vector or the input data (Data_in), which can be either the plaintext for encryption or the ciphertext for the decryption algorithm of the mode. The tweaks are generated by an LFSR. A tag register is used to accumulate the sums of plaintext blocks and the encryption of all-zero vectors (note that the plaintext blocks are obtained as outputs of the block cipher DEC operation when the decryption algorithm of the mode is performed). The part in purple background is additionally needed for tag verification after all the ciphertext blocks have been processed.

It can be seen that the LFSR that produces the tweak lies in the critical path of the circuit. Replacing this with a decimal up-counter (as in the specification presented in Figure 12) would increase the latency slightly since the delay of the incrementer used in

the counter is known to have circuit depth logarithmic in the size of the counter in bits. Table 19 tabulates the performance results of the circuit in Figure 13 when implemented with the Nangate 15 nm standard cell library, along with AES-256-GCM. For long messages, the the circuit processes one block per clock cycle on average, and hence theoretically, the maximum throughput the circuit can offer is 128 bits per latency period of the circuit. As can be seen, the circuit can reach a throughput close to 0.2 Tbps.

7 Conclusion

We have proposed Dialga, a family of low-latency tweakable block ciphers with 128/256-bit tweaks and 256-bit keys. Dialga minimizes latency using multiple linear layers with efficient cell permutations, optimal S-box selection via SAT evaluation, and a reflection tweakey schedule. Our comprehensive hardware benchmark showed that Dialga achieves nearly half the delay of QARMAv2, the most relevant competitor, and approximately a 40% reduction in area while maintaining the same security claims.

Acknowledgments

This work was supported by JST CRONOS, Japan, Grant Number JPMJCS25N1. Additional support was provided by JST AIP Acceleration Research, Japan, Grant Number JPMJCR24U1; JST START Project Promotion Type (Supporting Small Business Innovation Research (SBIR) Phase 1), Japan, Grant Number JPMJST2552 and JSPS KAKENHI, Grant Number JP24H00696.

References

[ABC+24] Ravi Anand, Subhadeep Banik, Andrea Caforio, Tatsuya Ishikawa, Takanori Isobe, Fukang Liu, Kazuhiko Minematsu, Mostafizar Rahman, and Kosei Sakamoto. Gleeok: A family of low-latency PRFs and its applications to authenticated encryption. *IACR TCHES*, 2024(2):545–587, 2024.

[ABD+23] Roberto Avanzi, Subhadeep Banik, Orr Dunkelman, Maria Eichlseder, Shibam Ghosh, Marcel Nageler, and Francesco Regazzoni. The QARMAv2 family of tweakable block ciphers. *IACR Trans. Symm. Cryptol.*, 2023(3):25–73, 2023.

[ABI+18] Gianira N. Alfarano, Christof Beierle, Takanori Isobe, Stefan Kölbl, and Gregor Leander. Shiftrows alternatives for aes-like ciphers and optimal cell permutations for midori and skinny. *IACR Trans. Symmetric Cryptol.*, 2018(2):20–47, 2018.

[AES01] Advanced Encryption Standard (AES). National Institute of Standards and Technology, NIST FIPS PUB 197, U.S. Department of Commerce, November 2001.

[AKM+24] Zahra Ahmadian, Akram Khalesi, Dounia M'foukh, Hossein Moghimi, and María Naya-Plasencia. Truncated differential cryptanalysis: New insights and application to QARMAv1-n and QARMAv2-64. *Des. Codes Cryptogr.*, 2024.

[Ava17] Roberto Avanzi. The QARMA block cipher family. *IACR Trans. Symm. Cryptol.*, 2017(1):4–44, 2017.

[Ava22] Roberto Avanzi. Cryptographic Protection of Random Access Memory: How Inconspicuous can Hardening Against the most Powerful Adversaries be? In *CCSW*, page 1. ACM, 2022.

[BBI+15] Subhadeep Banik, Andrey Bogdanov, Takanori Isobe, Kyoji Shibutani, Harunaga Hiwatari, Toru Akishita, and Francesco Regazzoni. Midori: A block cipher for low energy. In Tetsu Iwata and Jung Hee Cheon, editors, *Advances in Cryptology - ASIACRYPT 2015 - 21st International Conference on the Theory and Application of Cryptology and Information Security, Auckland, New Zealand, November 29 - December 3, 2015, Proceedings, Part II*, volume 9453 of *Lecture Notes in Computer Science*, pages 411–436. Springer, 2015.

[BBS99] Eli Biham, Alex Biryukov, and Adi Shamir. Cryptanalysis of skipjack reduced to 31 rounds using impossible differentials. In *EUROCRYPT*, volume 1592 of *Lecture Notes in Computer Science*, pages 12–23. Springer, 1999.

[BCG+12] Julia Borghoff, Anne Canteaut, Tim Güneysu, Elif Bilge Kavun, Miroslav Knežević, Lars R. Knudsen, Gregor Leander, Ventzislav Nikov, Christof Paar, Christian Rechberger, Peter Rombouts, Søren S. Thomsen, and Tolga Yalçin. PRINCE - A low-latency block cipher for pervasive computing applications - extended abstract. In Xiaoyun Wang and Kazue Sako, editors, *ASIACRYPT 2012*, volume 7658 of *LNCS*, pages 208–225. Springer, Berlin, Heidelberg, December 2012.

[BCLR17] Christof Beierle, Anne Canteaut, Gregor Leander, and Yann Rotella. Proving resistance against invariant attacks: How to choose the round constants. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology - CRYPTO 2017 - 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part II*, volume 10402 of *Lecture Notes in Computer Science*, pages 647–678. Springer, 2017.

[BDBN23] Christina Boura, Nicolas David, Rachelle Heim Boissier, and María Naya-Plasencia. Better steady than speedy: Full break of SPEEDY-7-192. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part IV*, volume 14007 of *LNCS*, pages 36–66. Springer, Cham, April 2023.

[BDD+23a] Yanis Belkheyar, Joan Daemen, Christoph Dobraunig, Santosh Ghosh, and Shahram Rasoolzadeh. BipBip: A low-latency tweakable block cipher with small dimensions. *IACR TCHES*, 2023(1):326–368, 2023.

[BDD+23b] Yanis Belkheyar, Joan Daemen, Christoph Dobraunig, Santosh Ghosh, and Shahram Rasoolzadeh. Introducing two low-latency cipher families: Sonic and supersonic. *IACR Cryptol. ePrint Arch.*, page 878, 2023.

[BEK+20] Dusan Bozilov, Maria Eichlseder, Miroslav Knezevic, Baptiste Lambin, Gregor Leander, Thorben Moos, Ventzislav Nikov, Shahram Rasoolzadeh, Yosuke Todo, and Friedrich Wiemer. PRINCEv2 - more security for (almost) no overhead. In Orr Dunkelman, Michael J. Jacobson Jr., and Colin O'Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 483–511. Springer, Cham, October 2020.

[Bey23] Tim Beyne. An invariant of the round function of qarmav2-64. *IACR Cryptol. ePrint Arch.*, page 963, 2023.

[BII+25] Subhadeep Banik, Tatsuya Ishikawa, Takanori Isobe, Ryoma Ito, Kazuhiko Minematsu, Kazuma Nakata, Mostafizar Rahman, and Kosei Sakamoto. Dialga: A family of low-latency tweakable block ciphers using multiple linear layers. *IACR Trans. Symmetric Cryptol.*, 2025(4):70–124, 2025.

[BIL+21] Subhadeep Banik, Takanori Isobe, Fukang Liu, Kazuhiko Minematsu, and Kosei Sakamoto. Orthros: A low-latency PRF. *IACR Trans. Symm. Cryptol.*, 2021(1):37–77, 2021.

[BJK+16] Christof Beierle, Jérémy Jean, Stefan Kölbl, Gregor Leander, Amir Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, and Siang Meng Sim. The SKINNY family of block ciphers and its low-latency variant MANTIS. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 123–153. Springer, Berlin, Heidelberg, August 2016.

[BK09] Alex Biryukov and Dmitry Khovratovich. Related-Key Cryptanalysis of the Full AES-192 and AES-256. In Mitsuru Matsui, editor, *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6-10, 2009. Proceedings*, volume 5912 of *Lecture Notes in Computer Science*, pages 1–18. Springer, 2009.

[BKL+07] Andrey Bogdanov, Lars R. Knudsen, Gregor Leander, Christof Paar, Axel Poschmann, Matthew J. B. Robshaw, Yannick Seurin, and C. Vikkelsoe. PRESENT: an ultra-lightweight block cipher. In Pascal Paillier and Ingrid Verbauwhede, editors, *Cryptographic Hardware and Embedded Systems - CHES 2007, 9th International Workshop, Vienna, Austria, September 10-13, 2007, Proceedings*, volume 4727 of *Lecture Notes in Computer Science*, pages 450–466. Springer, 2007.

[BL23] Augustin Bariant and Gaëtan Leurent. Truncated boomerang attacks and application to aes-based ciphers. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part IV*, volume 14007 of *Lecture Notes in Computer Science*, pages 3–35. Springer, 2023.

[BNN+12] Begül Bilgin, Svetla Nikova, Ventzislav Nikov, Vincent Rijmen, and Georg Stütz. Threshold implementations of all 3 ×3 and 4 ×4 s-boxes. In Emmanuel Prouff and Patrick Schaumont, editors, *Cryptographic Hardware and Embedded Systems - CHES 2012 - 14th International Workshop, Leuven, Belgium, September 9-12, 2012. Proceedings*, volume 7428 of *Lecture Notes in Computer Science*, pages 76–91. Springer, 2012.

[BS91] Eli Biham and Adi Shamir. Differential cryptanalysis of des-like cryptosystems. *J. Cryptol.*, 4(1):3–72, 1991.

[CGL+23] Federico Canale, Tim Güneysu, Gregor Leander, Jan Philipp Thoma, Yosuke Todo, and Rei Ueno. SCARF - A Low-Latency Block Cipher for Secure Cache-Randomization. In *USENIX Security Symposium*, pages 1937–1954. USENIX Association, 2023.

[CHP+18] Carlos Cid, Tao Huang, Thomas Peyrin, Yu Sasaki, and Ling Song. Boomerang Connectivity Table: A New Cryptanalysis Tool. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part II*, volume 10821 of *Lecture Notes in Computer Science*, pages 683–714. Springer, 2018.

[CJPS24] Benoît Cogliati, Jérémy Jean, Thomas Peyrin, and Yannick Seurin. A long tweak goes a long way: High multi-user security authenticated encryption from tweakable block ciphers. *IACR Communications in Cryptology*, 1(2), 2024.

[CXTQ23] Yaxin Cui, Hong Xu, Lin Tan, and Wenfeng Qi. SAT-aided differential cryptanalysis of lightweight block ciphers midori, MANTIS and QARMA. In Ding Wang, Moti Yung, Zheli Liu, and Xiaofeng Chen, editors, *ICICS 23*, volume 14252 of *LNCS*, pages 3–18. Springer, Singapore, November 2023.

[DDV20] Stéphanie Delaune, Patrick Derbez, and Mathieu Vavrille. Catching the fastest boomerangs application to SKINNY. *IACR Trans. Symmetric Cryptol.*, 2020(4):104–129, 2020.

[DEFN22] Patrick Derbez, Marie Euler, Pierre-Alain Fouque, and Phuong Hoa Nguyen. Revisiting related-key boomerang attacks on AES using computer-aided tool. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part III*, volume 13793 of *Lecture Notes in Computer Science*, pages 68–88. Springer, 2022.

[Din15] Itai Dinur. Cryptanalytic time-memory-data tradeoffs for FX-constructions with applications to PRINCE and PRIDE. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 231–253. Springer, Berlin, Heidelberg, April 2015.

[DKS10] Orr Dunkelman, Nathan Keller, and Adi Shamir. A Practical-Time Related-Key Attack on the KASUMI Cryptosystem Used in GSM and 3G Telephony. In Tal Rabin, editor, *Advances in Cryptology - CRYPTO 2010, 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010. Proceedings*, volume 6223 of *Lecture Notes in Computer Science*, pages 393–410. Springer, 2010.

[DKS14] Orr Dunkelman, Nathan Keller, and Adi Shamir. A Practical-Time Related-Key Attack on the KASUMI Cryptosystem Used in GSM and 3G Telephony. *J. Cryptol.*, 27(4):824–849, 2014.

[DO23] Siemen Dhooghe and Artemii Ovchinnikov. Threshold implementations with non-uniform inputs. In Claude Carlet, Kalikinkar Mandal, and Vincent Rijmen, editors, *Selected Areas in Cryptography - SAC 2023 - 30th International Conference, Fredericton, Canada, August 14-18, 2023, Revised Selected Papers*, volume 14201 of *Lecture Notes in Computer Science*, pages 97–123. Springer, 2023.

[DP15] Patrick Derbez and Léo Perrin. Meet-in-the-middle attacks and structural analysis of round-reduced PRINCE. In Gregor Leander, editor, *FSE 2015*, volume 9054 of *LNCS*, pages 190–216. Springer, Berlin, Heidelberg, March 2015.

[Dwo10] Morris Dworkin. Recommendation for Block Cipher Modes of Operation: The XTS-AES Mode for Confidentiality on Storage Devices. NIST Special Publication (SP) 800-38E, 2010.

[FLL+25] Antonio Flórez-Gutiérrez, Eran Lambooij, Gaëtan Leurent, Håvard Raddum, Tyge Tiessen, and Michiel Verbauwhede. Cryptanalysis of full SCARF. In

Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part I*, volume 15601 of *Lecture Notes in Computer Science*, pages 397–426. Springer, 2025.

[GIK+19] Chun Guo, Tetsu Iwata, Mustafa Khairallah, Kazuhiko Minematsu, and Thomas Peyrin. Romulus. A submission to NIST Lightweight Cryptography, 2019.

[GJN+16] Jian Guo, Jérémy Jean, Ivica Nikolic, Kexin Qiao, Yu Sasaki, and Siang Meng Sim. Invariant subspace attack against midori64 and the resistance criteria for s-box designs. *IACR Trans. Symmetric Cryptol.*, 2016(1):33–56, 2016.

[GMW24] Patricia Greene, Mark Motley, and Bryan Weeks. ARADI and LLAMA: low-latency cryptography for memory encryption. *IACR Cryptol. ePrint Arch.*, page 1240, 2024.

[GR16] Lorenzo Grassi and Christian Rechberger. Practical low data-complexity subspace-trail cryptanalysis of round-reduced PRINCE. In Orr Dunkelman and Somitra Kumar Sanadhya, editors, *INDOCRYPT 2016*, volume 10095 of *LNCS*, pages 322–342. Springer, Cham, December 2016.

[Gue16] Shay Gueron. Memory Encryption for General-Purpose Processors. *IEEE Secur. Priv.*, 14(6):54–62, 2016.

[HGSE24] Hosein Hadipour, Simon Gerhalter, Sadegh Sadeghi, and Maria Eichlseder. Improved search for integral, impossible differential and zero-correlation attacks application to ascon, forkskinny, skinny, mantis, PRESENT and qarmav2. *IACR Trans. Symmetric Cryptol.*, 2024(1):234–325, 2024.

[HKPT24] Kai Hu, Mustafa Khairallah, Thomas Peyrin, and Quan Quan Tan. uKNIT: Breaking round-alignment for cipher design – featuring uKNIT-BC, an ultra low-latency block cipher. Cryptology ePrint Archive, Paper 2024/1962, 2024.

[HT24] Hosein Hadipour and Yosuke Todo. Cryptanalysis of qarmav2. *IACR Trans. Symmetric Cryptol.*, 2024(1):188–213, 2024.

[IMO+22] Akiko Inoue, Kazuhiko Minematsu, Maya Oda, Rei Ueno, and Naofumi Homma. ELM: A Low-Latency and Scalable Memory Encryption Scheme. *IEEE Trans. Inf. Forensics Secur.*, 17:2628–2643, 2022.

[JNP+14] Jérémy Jean, Ivica Nikolic, Thomas Peyrin, Lei Wang, and Shuang Wu. Security analysis of PRINCE. In Shiho Moriai, editor, *FSE 2013*, volume 8424 of *LNCS*, pages 92–111. Springer, Berlin, Heidelberg, March 2014.

[Kar08] Orhun Kara. Reflection cryptanalysis of some ciphers. In Dipanwita Roy Chowdhury, Vincent Rijmen, and Abhijit Das, editors, *Progress in Cryptology - INDOCRYPT 2008, 9th International Conference on Cryptology in India, Kharagpur, India, December 14-17, 2008. Proceedings*, volume 5365 of *Lecture Notes in Computer Science*, pages 294–307. Springer, 2008.

[KDK+14] Yoongu Kim, Ross Daly, Jeremie S. Kim, Chris Fallin, Ji-Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors. In *ISCA*, pages 361–372. IEEE Computer Society, 2014.

[KGGY20] Andrew Kwong, Daniel Genkin, Daniel Gruss, and Yuval Yarom. RAMBleed: Reading bits in memory without accessing them. In *2020 IEEE Symposium on Security and Privacy*, pages 695–711. IEEE Computer Society Press, May 2020.

[KLPR10] Lars R. Knudsen, Gregor Leander, Axel Poschmann, and Matthew J. B. Robshaw. Printcipher: A block cipher for ic-printing. In Stefan Mangard and François-Xavier Standaert, editors, *Cryptographic Hardware and Embedded Systems, CHES 2010, 12th International Workshop, Santa Barbara, CA, USA, August 17-20, 2010. Proceedings*, volume 6225 of *Lecture Notes in Computer Science*, pages 16–32. Springer, 2010.

[Knu94] Lars R. Knudsen. Truncated and higher order differentials. In Bart Preneel, editor, *Fast Software Encryption: Second International Workshop. Leuven, Belgium, 14-16 December 1994, Proceedings*, volume 1008 of *Lecture Notes in Computer Science*, pages 196–211. Springer, 1994.

[KR11] Ted Krovetz and Phillip Rogaway. The software performance of authenticated-encryption modes. In Antoine Joux, editor, *FSE 2011*, volume 6733 of *LNCS*, pages 306–327. Springer, Berlin, Heidelberg, February 2011.

[LAAZ11] Gregor Leander, Mohamed Ahmed Abdelraheem, Hoda AlKhzaimi, and Erik Zenner. A cryptanalysis of printcipher: The invariant subspace attack. In Phillip Rogaway, editor, *Advances in Cryptology - CRYPTO 2011 - 31st Annual Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2011. Proceedings*, volume 6841 of *Lecture Notes in Computer Science*, pages 206–221. Springer, 2011.

[LHW19] Muzhou Li, Kai Hu, and Meiqin Wang. Related-tweak statistical saturation cryptanalysis and its application on QARMA. *IACR Trans. Symm. Cryptol.*, 2019(1):236–263, 2019.

[LMMR21] Gregor Leander, Thorben Moos, Amir Moradi, and Shahram Rasoolzadeh. The SPEEDY family of block ciphers engineering an ultra low-latency cipher from gate level for secure processor architectures. *IACR TCHES*, 2021(4):510–545, 2021. https://tches.iacr.org/index.php/TCHES/article/view/9074.

[LSW22] Muzhou Li, Ling Sun, and Meiqin Wang. Automated key recovery attacks on round-reduced orthros. In Lejla Batina and Joan Daemen, editors, *AFRICACRYPT 22*, volume 2022 of *LNCS*, pages 189–213. Springer, Cham, July 2022.

[Mat93] Mitsuru Matsui. Linear cryptanalysis method for DES cipher. In Tor Helleseth, editor, *Advances in Cryptology - EUROCRYPT '93, Workshop on the Theory and Application of of Cryptographic Techniques, Lofthus, Norway, May 23-27, 1993, Proceedings*, volume 765 of *Lecture Notes in Computer Science*, pages 386–397. Springer, 1993.

[MMR+15] Mayler Martins, Jody Maick Matos, Renato P. Ribas, Andr'e Reis, Guilherme Schlinker, Lucio Rech, and Jens Michelsen. Open cell library in 15nm freepdk technology. In *ISPD 2015*, page 171–178, New York, NY, USA, 2015. Association for Computing Machinery.

[Mur11] Sean Murphy. The Return of the Cryptographic Boomerang. *IEEE Trans. Inf. Theory*, 57(4):2517–2521, 2011.

[nis17] NIST Lightweight Cryptography, 2017. Accessed: 2024-07-15.

[NSS20] Yusuke Naito, Yu Sasaki, and Takeshi Sugawara. Lightweight authenticated encryption mode suitable for threshold implementation. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 705–735. Springer, Cham, May 2020.

[NSS22] Yusuke Naito, Yu Sasaki, and Takeshi Sugawara. Secret can be public: Low-memory AEAD mode for high-order masking. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 315–345. Springer, Cham, August 2022.

[PSS24] Thomas Peters, Yaobin Shen, and François-Xavier Standaert. Multiplex: TBC-based Authenticated Encryption with Sponge-Like Rate. *Transactions on Symmetric-key Cryptology*, 2:1–34, 2024.

[QDW+22] Lingyue Qin, Xiaoyang Dong, Anyu Wang, Jialiang Hua, and Xiaoyun Wang. Mind the TWEAKEY schedule: Cryptanalysis on SKINNYe-64-256. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part I*, volume 13791 of *LNCS*, pages 287–317. Springer, Cham, December 2022.

[RS22] Raghvendra Rohit and Santanu Sarkar. Cryptanalysis of reduced round SPEEDY. In Lejla Batina and Joan Daemen, editors, *AFRICACRYPT 22*, volume 2022 of *LNCS*, pages 133–149. Springer, Cham, July 2022.

[SBY+13] Hadi Soleimany, Céline Blondeau, Xiaoli Yu, Wenling Wu, Kaisa Nyberg, Huiling Zhang, Lei Zhang, and Yanfeng Wang. Reflection cryptanalysis of prince-like ciphers. In Shiho Moriai, editor, *Fast Software Encryption - 20th International Workshop, FSE 2013, Singapore, March 11-13, 2013. Revised Selected Papers*, volume 8424 of *Lecture Notes in Computer Science*, pages 71–91. Springer, 2013.

[SBY+14] Hadi Soleimany, Céline Blondeau, Xiaoli Yu, Wenling Wu, Kaisa Nyberg, Huiling Zhang, Lei Zhang, and Yanfeng Wang. Reflection cryptanalysis of PRINCE-like ciphers. In Shiho Moriai, editor, *FSE 2013*, volume 8424 of *LNCS*, pages 71–91. Springer, Berlin, Heidelberg, March 2014.

[SBY+15] Hadi Soleimany, Céline Blondeau, Xiaoli Yu, Wenling Wu, Kaisa Nyberg, Huiling Zhang, Lei Zhang, and Yanfeng Wang. Reflection cryptanalysis of prince-like ciphers. *J. Cryptol.*, 28(3):718–744, 2015.

[SCS23] Soumya Sahoo, Debasmita Chakraborty, and Santanu Sarkar. Unleashing the power of differential fault attacks on qarmav2. *IACR Cryptol. ePrint Arch.*, page 1667, 2023.

[SII23] Kosei Sakamoto, Ryoma Ito, and Takanori Isobe. Parallel SAT framework to find clustering of differential characteristics and its applications. In Claude Carlet, Kalikinkar Mandal, and Vincent Rijmen, editors, *Selected Areas in Cryptography - SAC 2023 - 30th International Conference, Fredericton, Canada, August 14-18, 2023, Revised Selected Papers*, volume 14201 of *Lecture Notes in Computer Science*, pages 409–428. Springer, 2023.

[SPS+22] Yaobin Shen, Thomas Peters, François-Xavier Standaert, Gaëtan Cassiers, and Corentin Verhamme. Triplex: an efficient and one-pass leakage-resistant mode of operation. *IACR TCHES*, 2022(4):135–162, 2022.

[SWW18] Ling Sun, Wei Wang, and Meiqin Wang. More accurate differential properties of LED64 and midori64. *IACR Trans. Symmetric Cryptol.*, 2018(3):93–123, 2018.

[TISI23] Kazuma Taka, Tatsuya Ishikawa, Kosei Sakamoto, and Takanori Isobe. An efficient strategy to construct a better differential on multiple-branch-based designs: Application to orthros. In Mike Rosulek, editor, *CT-RSA 2023*, volume 13871 of *LNCS*, pages 277–304. Springer, Cham, April 2023.

[Tod15] Yosuke Todo. Structural evaluation by generalized integral property. In *EUROCRYPT (1)*, volume 9056 of *Lecture Notes in Computer Science*, pages 287–314. Springer, 2015.

[Wag99] David A. Wagner. The Boomerang Attack. In Lars R. Knudsen, editor, *Fast Software Encryption, 6th International Workshop, FSE '99, Rome, Italy, March 24-26, 1999, Proceedings*, volume 1636 of *Lecture Notes in Computer Science*, pages 156–170. Springer, 1999.

[WBD+24] Jinliang Wang, Christina Boura, Patrick Derbez, Kai Hu, Muzhou Li, and Meiqin Wang. Cryptanalysis of full-round bipbip. *IACR Trans. Symmetric Cryptol.*, 2024(2):68–84, 2024.

[WHWL24] Jianhua Wang, Tao Huang, Shuang Wu, and Zilong Liu. Twinkle: A family of low-latency schemes for authenticated encryption and pointer authentication. *IACR Commun. Cryptol.*, 1(2):20, 2024.

[WNL+23] Jinliang Wang, Chao Niu, Qun Liu, Muzhou Li, Bart Preneel, and Meiqin Wang. Cryptanalysis of SPEEDY. In Leonie Simpson and Mir Ali Rezazadeh Baee, editors, *ACISP 23*, volume 13915 of *LNCS*, pages 124–156. Springer, Cham, July 2023.

[XZBL16] Zejun Xiang, Wentao Zhang, Zhenzhen Bao, and Dongdai Lin. Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers. In *ASIACRYPT (1)*, volume 10031 of *Lecture Notes in Computer Science*, pages 648–678, 2016.

A Algorithm

Algorithm 1 Encryption of Dialga with a 128-bit tweak key(P, T, K, α, β)

function DIALGA-128(P, K, T, α, β)
  $(K_0||K_1) \leftarrow K$
  $S_d \leftarrow P$
  $S_t \leftarrow T$
  $S_d, S_t \leftarrow R_f(S_d, S_t, K_0, K_1, \alpha)$
  $S_d, S_t \leftarrow R_m(S_d, S_t, K_0, K_1, \alpha)$
  $S_d, S_t \leftarrow R_b(S_d, S_t, K_0, K_1, \alpha, \beta)$
  return S_d
end function
function $R_f(S_d, S_t, K_0, K_1, \alpha)$
  $S_d \leftarrow S_d \oplus S_t \oplus K_0 \oplus K_1$
  for $i = 1$ to α **do**
    if $i = 1$ **then**
      $S_t \leftarrow S_t \oplus K_{(i-1)\%2}$
    else
      $S_t \leftarrow R_{(i-1)\%4}(S_t) \oplus K_{(i-1)\%2}$
    end if
    $S_d \leftarrow R_{(2i-2)\%4}(S_d) \oplus K_{i\%2} \oplus c^f_{2(i-1)}$
    $S_d \leftarrow R_{(2i-1)\%4}(S_d) \oplus S_t \oplus c^f_{2i-1}$
  end for
  return S_d, S_t
end function
function $R_m(S_d, S_t, K_0, K_1, \alpha)$
  $S_t \leftarrow \mathsf{SubCell}(S_t)$
  $S_d \leftarrow R_{2\alpha\%4}(S_d) \oplus K_{(\alpha+1)\%2} \oplus c^m_0 \oplus S_t$
  $S_t \leftarrow \mathsf{SubCell}^{-1}(S_t) \oplus K_{(\alpha-1)\%2}$
  $S_t \leftarrow R^{-1}_{(\alpha-1)\%4}(S_t)$
  $S_d \leftarrow R_{(2\alpha+1)\%4}(S_d) \oplus MS(S_t) \oplus c^m_1$
  return S_d, S_t
end function
function $R_b(S_d, S_t, K_0, K_1, \alpha, \beta)$
  for $i = 1$ to β **do**
    if $i = \beta$ **then**
      $S_t \leftarrow S_t \oplus K_{(\alpha-i-1)\%2}$
    else
      $S_t \leftarrow R^{-1}_{(\alpha-i-1)\%4}(S_t \oplus K_{(\alpha-i-1)\%2})$
    end if
    $S_d \leftarrow R_{2(\alpha+i)\%4}(S_d) \oplus K_{(\alpha+i+1)\%2} \oplus c^b_{2(i-1)}$
    $S_d \leftarrow R_{(2(\alpha+i)+1)\%4}(S_d) \oplus MS(S_t) \oplus c^b_{2i-1}$
  end for
  $S_d \leftarrow \mathsf{SubCell}(S_d)$
  $S_d \leftarrow S_d \oplus K_0 \oplus K_1$
  return S_d, S_t
end function

Algorithm 2 Encryption of Dialga with a 256-bit tweak key(P, T, K, α, β)

function DIALGA-256(P, K, T, α, β)
  $(K_0||K_1) \leftarrow K$
  $(T_0||T_1) \leftarrow T$
  $S_d \leftarrow P$
  $S_{t_0}, S_{t_1} \leftarrow T_0, T_1$
  $S_d, S_{t_0}, S_{t_1} \leftarrow R_f(S_d, S_{t_0}, S_{t_1}, K_0, K_1, \alpha)$
  $S_d, S_{t_0}, S_{t_1} \leftarrow R_m(S_d, S_{t_0}, S_{t_1}, K_0, K_1, \alpha)$
  $S_d, S_{t_0}, S_{t_1} \leftarrow R_b(S_d, S_{t_0}, S_{t_1}, K_0, K_1, \alpha, \beta)$
  return S_d
end function
function $R_f(S_d, S_{t_0}, S_{t_1}, K_0, K_1, \alpha)$
  $S_d \leftarrow S_d \oplus S_{t_0} \oplus K_0 \oplus K_1$
  for $i = 1$ to α **do**
    if $i = 1$ **then**
      $S_{t_0} \leftarrow S_{t_0} \oplus K_{(i-1)\%2}$
      $S_d \leftarrow R_{(2i-2)\%4}(S_d) \oplus S_{t_1} \oplus K_{i\%2} \oplus c^f_{2(i-1)}$
      $S_d \leftarrow R_{(2i-1)\%4}(S_d) \oplus S_{t_0} \oplus c^f_{2i-1}$
      $S_{t_1} \leftarrow S_{t_1} \oplus K_{i\%2}$
    else
      $S_{t_0} \leftarrow R_{(i-1)\%4}(S_{t_0}) \oplus K_{(i-1)\%2}$
      $S_d \leftarrow R_{(2i-2)\%4}(S_d) \oplus S_{t_1} \oplus c^f_{2(i-1)}$
      $S_d \leftarrow R_{(2i-1)\%4}(S_d) \oplus S_{t_0} \oplus c^f_{2i-1}$
      $S_{t_1} \leftarrow R_{(i-1)\%4}(S_{t_1}) \oplus K_{i\%2}$
    end if
  end for
  return S_d, S_{t_0}, S_{t_1}
end function
function $R_m(S_d, S_{t_0}, S_{t_1}, K_0, K_1, \alpha)$
  $\alpha \leftarrow i$
  $S_{t_0} \leftarrow \mathsf{SubCell}(S_{t_0})$
  $S_d \leftarrow R_{2\alpha\%4}(S_d) \oplus S_{t_1} \oplus c^m_0 \oplus S_{t_0}$
  $S_{t_0} \leftarrow \mathsf{SubCell}^{-1}(S_{t_0}) \oplus K_{(\alpha-1)\%2}$
  $S_{t_0} \leftarrow R^{-1}_{(v-1)\%4}(S_{t_0})$
  $S_d \leftarrow R_{(2\alpha+1)\%4}(S_d) \oplus MS(S_{t_0}) \oplus c^m_1$
  $S_{t_1} \leftarrow R^{-1}_{(\alpha-1)\%4}(S_{t_1} \oplus K_{\alpha\%2})$
  return S_d, S_{t_0}, S_{t_1}
end function
function $R_b(S_d, S_{t_0}, S_{t_1}, K_0, K_1, \alpha, \beta)$
  for $i = 1$ to β **do**
    if $i = \beta$ **then**
      $S_{t_0} \leftarrow S_{t_0} \oplus K_{(\alpha-i-1)\%2}$
      $S_d \leftarrow R_{2(\alpha+i)\%4}(S_d) \oplus MS(S_{t_1}) \oplus c^b_{2(i-1)}$
      $S_d \leftarrow R_{(2(\alpha+i)+1)\%4}(S_d) \oplus MS(S_{t_0}) \oplus c^b_{2i-1}$
      $S_{t_1} \leftarrow S_{t_1} \oplus K_{(\alpha-i)\%2}$
    else
      $S_{t_0} \leftarrow R^{-1}_{(\alpha-i-1)\%4}(S_{t_0} \oplus K_{(\alpha-i-1)\%2})$
      $S_d \leftarrow R_{2(\alpha+i)\%4}(S_d) \oplus MS(S_{t_1}) \oplus c^b_{2(i-1)}$
      $S_d \leftarrow R_{(2(\alpha+i)+1)\%4}(S_d) \oplus MS(S_{t_0}) \oplus c^b_{2i-1}$
      $S_{t_1} \leftarrow R^{-1}_{(\alpha-i-1)\%4}(S_{t_1} \oplus K_{(\alpha-i)\%2})$
    end if
  end for
  $S_d \leftarrow \mathsf{SubCell}(S_d)$
  $S_d \leftarrow S_d \oplus MS(S_{t_1}) \oplus K_0 \oplus K_1$
  return S_d, S_{t_0}, S_{t_1}
end function

B Round Functions with Multiple Linear Layers

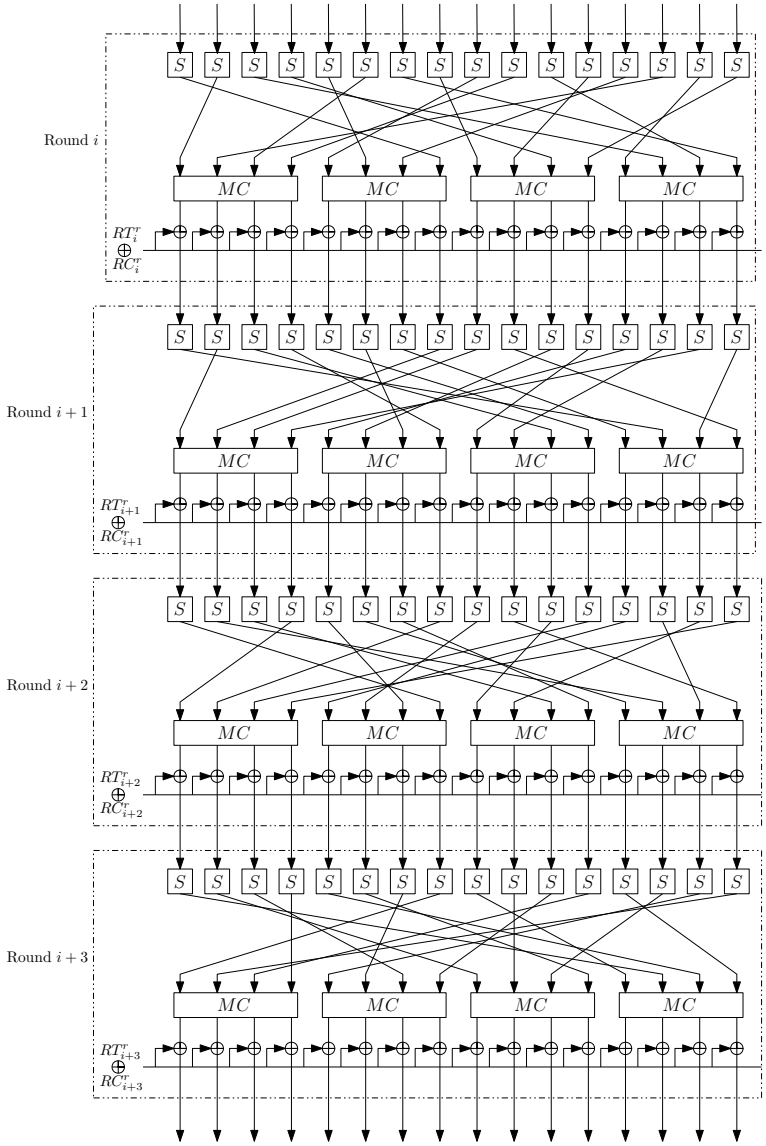

Figure 14: Round function of Dialga.

C Lower Bounds of Number of Active S-boxes in Related-Tweak Setting

Tables 20–23 show the lower bounds of the number of active S-boxes in the related-tweak setting, excluding the initial round function of the tweakey schedule to provide the results when merely searching for a distinguisher.

In our experiments, we have observed how differences in the starting point of the round function (R_i) in the data processing (DP) and tweakey schedule (TS) parts affect the lower bounds of the number of active S-boxes. The columns of "i in DP" and "i in TS" in these tables indicate the starting point of R_i in the DP and TS parts, which is R_0 and R_1 in our design, respectively.

As a result, although there are some differences, the impact of the different starting points is not considered to be significant.

Table 20: Lower bounds of the number of active S-boxes in the related-tweak setting of Dialga-128†, excluding the initial round function of the tweakey schedule to provide the results when merely searching for a distinguisher. The numbers in parentheses show an example of the individual number of active S-boxes in the data processing and tweakey scheduling parts $[\#AS_d/\#AS_t]$.

Starting point of R_i		# of rounds											
i in DP	i in TS	4	5	6	7	8	9	10	11	12	13	14	15
0	0	6 [4/2]	10 [4/6]	13 [4/9]	21 [11/10]	34 [24/10]	36 [26/10]	**52** [52/0]	57 [38/19]	59 [33/26]	63 [36/27]	67 [41/26]	– –
	1	6 [4/2]	10 [4/6]	13 [4/9]	22 [13/9]	34 [24/10]	36 [26/10]	**50** [31/19]	58 [39/19]	62 [36/26]	65 [46/19]	– –	– –
	2	6 [4/2]	10 [4/6]	13 [4/9]	22 [13/9]	34 [24/10]	36 [26/10]	**50** [31/19]	56 [37/19]	60 [36/24]	63 [44/19]	66 [47/19]	– –
	3	6 [4/2]	12 [10/2]	22 [16/6]	24 [18/6]	36 [24/12]	**40** [28/12]	52 [52/0]	59 [59/0]	60 [34/26]	66 [40/26]	– –	– –
1	0	6 [4/2]	10 [6/4]	22 [12/10]	26 [21/5]	36 [24/12]	**40** [30/10]	52 [52/0]	57 [57/0]	62 [32/30]	68 [38/30]	– –	– –
	1	6 [4/2]	10 [6/4]	20 [8/12]	24 [14/10]	36 [22/14]	**42** [32/10]	52 [22/30]	57 [57/0]	62 [35/27]	68 [36/32]	– –	– –
	2	6 [4/2]	10 [6/4]	22 [12/10]	26 [16/10]	36 [22/14]	**42** [28/14]	52 [52/0]	57 [57/0]	58 [30/28]	64 [36/28]	– –	– –
	3	6 [4/2]	10 [4/6]	13 [4/9]	22 [13/9]	36 [26/10]	**40** [22/18]	52 [52/0]	57 [38/19]	58 [28/30]	64 [34/30]	– –	– –
2	0	6 [4/2]	10 [4/6]	13 [4/9]	22 [13/9]	36 [24/12]	38 [20/18]	**48** [48/0]	55 [55/0]	60 [32/28]	64 [36/28]	– –	– –
	1	6 [4/2]	10 [4/6]	13 [4/9]	22 [13/9]	36 [26/10]	**40** [22/18]	48 [48/0]	55 [55/0]	58 [29/29]	62 [33/29]	66 [37/29]	– –
	2	6 [4/2]	10 [4/6]	13 [4/9]	21 [11/10]	36 [26/10]	**40** [22/18]	48 [48/0]	55 [55/0]	60 [32/28]	63 [38/25]	68 [43/25]	– –
	3	6 [4/2]	10 [6/4]	22 [12/10]	26 [16/10]	36 [26/10]	**43** [43/0]	48 [48/0]	55 [55/0]	64 [64/0]	– –	– –	– –
3	0	6 [4/2]	10 [4/6]	13 [4/9]	22 [13/9]	36 [24/12]	**40** [22/18]	48 [48/0]	56 [37/19]	61 [42/19]	64 [45/19]	– –	– –
	1	6 [4/2]	10 [4/6]	13 [4/9]	21 [11/10]	36 [26/10]	38 [20/18]	**48** [48/0]	56 [37/19]	60 [32/28]	64 [36/28]	– –	– –
	2	6 [4/2]	10 [4/6]	13 [4/9]	21 [11/10]	36 [26/10]	38 [20/18]	**48** [48/0]	56 [37/19]	61 [42/19]	65 [46/19]	– –	– –
	3	6 [4/2]	10 [6/4]	22 [12/10]	26 [16/10]	34 [22/12]	**41** [41/0]	48 [48/0]	57 [57/0]	60 [32/28]	66 [38/28]	– –	– –

Table 21: Lower bounds of the number of active S-boxes in the related-tweak setting of Dialga-128, excluding the initial round function of the tweakey schedule to provide the results when merely searching for a distinguisher. The numbers in parentheses show an example of the individual number of active S-boxes in the data processing and tweakey scheduling parts ($\#AS_d/\#AS_t$).

Starting point of R_i		# of rounds											
i in DP	i in TS	4	5	6	7	8	9	10	11	12	13	14	15
0	0	6 (4/2)	10 (4/6)	13 (4/9)	21 (11/10)	34 (24/10)	36 (26/10)	45 (26/19)	52 (33/19)	63 (37/26)	**69** (69/0)	– –	– –
	1	6 (4/2)	10 (4/6)	13 (4/9)	22 (13/9)	34 (24/10)	36 (26/10)	45 (26/19)	52 (33/19)	**64** (64/0)	– –	– –	– –
	2	6 (4/2)	10 (4/6)	13 (4/9)	22 (13/9)	34 (24/10)	36 (26/10)	45 (26/19)	52 (33/19)	**64** (64/0)	– –	– –	– –
	3	6 (4/2)	12 (10/2)	22 (16/6)	24 (18/6)	36 (24/12)	40 (28/12)	52 (52/0)	57 (33/24)	**64** (64/0)	– –	– –	– –
1	0	6 (4/2)	10 (6/4)	22 (12/10)	26 (21/5)	36 (24/12)	40 (30/10)	49 (30/19)	56 (37/19)	**64** (64/0)	– –	– –	– –
	1	6 (4/2)	10 (6/4)	20 (8/12)	24 (14/10)	36 (22/14)	42 (32/10)	51 (32/19)	56 (26/30)	**64** (64/0)	– –	– –	– –
	2	6 (4/2)	10 (6/4)	22 (12/10)	26 (16/10)	36 (22/14)	42 (28/14)	51 (32/19)	57 (57/0)	**64** (64/0)	– –	– –	– –
	3	6 (4/2)	10 (4/6)	13 (4/9)	22 (13/9)	36 (26/10)	40 (22/18)	47 (22/25)	52 (25/27)	63 (33/30)	**67** (37/30)	– –	– –
2	0	6 (4/2)	10 (4/6)	13 (4/9)	22 (13/9)	36 (24/12)	38 (20/18)	48 (48/0)	55 (55/0)	**64** (64/0)	– –	– –	– –
	1	6 (4/2)	10 (4/6)	13 (4/9)	22 (13/9)	36 (26/10)	40 (22/18)	48 (48/0)	55 (55/0)	**64** (64/0)	– –	– –	– –
	2	6 (4/2)	10 (4/6)	13 (4/9)	21 (11/10)	36 (26/10)	40 (22/18)	48 (48/0)	55 (55/0)	**64** (64/0)	– –	– –	– –
	3	6 (4/2)	10 (6/4)	22 (12/10)	26 (16/10)	36 (26/10)	43 (43/0)	48 (48/0)	54 (28/26)	**64** (64/0)	– –	– –	– –
3	0	6 (4/2)	10 (4/6)	13 (4/9)	22 (13/9)	36 (24/12)	40 (22/18)	48 (48/0)	56 (37/19)	**64** (64/0)	– –	– –	– –
	1	6 (4/2)	10 (4/6)	13 (4/9)	21 (11/10)	36 (26/10)	38 (20/18)	48 (48/0)	56 (34/22)	**64** (64/0)	– –	– –	– –
	2	6 (4/2)	10 (4/6)	13 (4/9)	21 (11/10)	36 (26/10)	38 (20/18)	48 (48/0)	56 (27/29)	**64** (64/0)	– –	– –	– –
	3	6 (4/2)	10 (6/4)	22 (12/10)	26 (16/10)	34 (22/12)	41 (41/0)	48 (48/0)	57 (57/0)	**64** (64/0)	– –	– –	– –

Table 22: Lower bounds of the number of active S-boxes in the related-tweak setting of Dialga-256†, excluding the initial round function of the tweakey schedule to provide the results when merely searching for a distinguisher. The numbers in parentheses show an example of the individual number of active S-boxes in the data processing and tweakey scheduling parts [$\#AS_d/\#AS_{t_0}/\#AS_{t_1}$].

Starting point of R_i		# of rounds											
i in DP	i in TS	4	5	6	7	8	9	10	11	12	13	14	15
0	0	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	21 [11/10/0]	29 [24/0/5]	36 [26/10/0]	**45** [31/0/14]	47 [33/0/14]	50 [40/0/10]	55 [45/0/10]	62 [48/0/14]	67 [53/0/14]
	1	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	22 [13/9/0]	27 [12/10/5]	36 [26/10/0]	**46** [32/0/14]	47 [33/0/14]	52 [42/0/10]	57 [47/0/10]	62 [48/0/14]	67 [53/0/14]
	2	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	22 [10/7/5]	27 [12/10/5]	36 [26/10/0]	**45** [31/0/14]	47 [33/0/14]	52 [42/0/10]	57 [47/0/10]	60 [46/0/14]	65 [51/0/14]
	3	5 [4/0/1]	9 [7/0/2]	15 [6/5/4]	22 [13/0/9]	31 [22/0/9]	**40** [28/12/0]	48 [36/0/12]	48 [36/0/12]	54 [30/0/24]	58 [34/0/24]	62 [38/0/24]	68 [44/0/24]
1	0	5 [4/0/1]	9 [7/0/2]	15 [13/0/2]	22 [13/0/9]	30 [24/0/6]	**40** [30/10/0]	44 [32/0/12]	47 [29/0/18]	52 [42/0/10]	57 [47/0/10]	62 [50/0/12]	67 [53/0/14]
	1	5 [4/0/1]	9 [7/0/2]	15 [13/0/2]	22 [13/0/9]	31 [26/0/5]	**42** [28/0/14]	47 [33/0/14]	47 [29/0/18]	54 [44/0/10]	58 [46/0/12]	62 [50/0/12]	67 [53/0/14]
	2	5 [4/0/1]	9 [7/0/2]	15 [13/0/2]	22 [13/0/9]	30 [20/0/10]	**41** [27/0/14]	46 [34/0/12]	46 [34/0/12]	52 [42/0/10]	57 [47/0/10]	64 [44/0/20]	– –
	3	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	20 [9/6/5]	31 [26/0/5]	**40** [22/18/0]	47 [33/0/14]	48 [30/0/18]	53 [43/0/10]	58 [48/0/10]	64 [50/0/14]	– –
2	0	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	22 [13/9/0]	31 [26/0/5]	38 [20/18/0]	**47** [33/0/14]	47 [33/0/14]	52 [42/0/10]	56 [44/0/12]	62 [50/0/12]	68 [56/0/12]
	1	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	22 [13/9/0]	25 [10/10/5]	**40** [22/18/0]	46 [34/0/12]	46 [34/0/12]	52 [40/0/12]	59 [49/0/10]	62 [46/0/16]	67 [53/0/14]
	2	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	21 [11/10/0]	30 [20/0/10]	38 [26/0/12]	**47** [33/0/14]	47 [33/0/14]	52 [42/0/10]	57 [47/0/10]	62 [48/0/14]	66 [54/0/12]
	3	5 [4/0/1]	9 [7/0/2]	15 [13/0/2]	23 [13/5/5]	31 [26/0/5]	**40** [30/0/10]	47 [33/0/14]	47 [33/0/14]	56 [40/0/16]	58 [42/0/16]	63 [49/0/14]	68 [48/0/20]
3	0	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	22 [13/0/9]	25 [10/10/5]	**40** [22/18/0]	45 [35/0/10]	45 [35/0/10]	48 [38/0/10]	53 [43/0/10]	62 [50/0/12]	68 [54/0/14]
	1	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	21 [11/10/0]	30 [25/0/5]	38 [20/18/0]	**45** [35/0/10]	45 [35/0/10]	48 [38/0/10]	53 [43/0/10]	62 [42/0/20]	66 [46/0/20]
	2	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	21 [11/10/0]	29 [14/10/5]	38 [20/18/0]	**45** [35/0/10]	45 [35/0/10]	48 [38/0/10]	53 [43/0/10]	62 [42/0/20]	66 [46/0/20]
	3	5 [4/0/1]	9 [7/0/2]	15 [13/0/2]	25 [15/5/5]	30 [24/0/6]	**41** [41/0/0]	46 [34/0/12]	47 [33/0/14]	54 [40/0/14]	60 [40/0/20]	64 [44/0/20]	– –

Table 23: Lower bounds of the number of active S-boxes in the related-tweak setting of Dialga-256, excluding the initial round function of the tweakey schedule to provide the results when merely searching for a distinguisher. The numbers in parentheses show an example of the individual number of active S-boxes in the data processing and tweakey scheduling parts $[\#AS_d/\#AS_{t_0}/\#AS_{t_1}]$.

Starting point of R_i		# of rounds											
i in DP	i in TS	4	5	6	7	8	9	10	11	12	13	14	15
0	0	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	21 [11/10/0]	29 [24/0/5]	36 [26/10/0]	45 [31/0/14]	52 [33/19/0]	62 [41/0/21]	**64** [41/0/23]	– –	– –
	1	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	22 [13/9/0]	27 [12/10/5]	36 [26/10/0]	45 [26/19/0]	52 [33/19/0]	60 [30/0/30]	60 [30/0/30]	**66** [36/0/30]	– –
	2	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	22 [10/7/5]	27 [12/10/5]	36 [26/10/0]	45 [26/19/0]	52 [33/19/0]	62 [37/0/25]	**64** [38/0/26]	– –	– –
	3	5 [4/0/1]	9 [7/0/2]	15 [6/5/4]	22 [13/0/9]	31 [22/0/9]	40 [28/12/0]	48 [36/0/12]	54 [26/0/28]	61 [33/0/28]	62 [35/0/27]	**70** [42/0/28]	– –
1	0	5 [4/0/1]	9 [7/0/2]	15 [13/0/2]	22 [13/0/9]	30 [24/0/6]	40 [30/10/0]	44 [32/0/12]	56 [37/19/0]	63 [36/0/27]	63 [36/0/27]	**66** [39/0/27]	– –
	1	5 [4/0/1]	9 [7/0/2]	15 [13/0/2]	22 [13/0/9]	31 [26/0/5]	42 [28/0/14]	47 [33/0/14]	56 [26/30/0]	63 [36/0/27]	**64** [41/0/23]	– –	– –
	2	5 [4/0/1]	9 [7/0/2]	15 [13/0/2]	22 [13/0/9]	30 [20/0/10]	41 [27/0/14]	46 [34/0/12]	57 [57/0/0]	63 [40/0/23]	**64** [38/0/26]	– –	– –
	3	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	20 [9/6/5]	31 [26/0/5]	40 [22/18/0]	47 [22/25/0]	52 [25/27/0]	60 [34/0/26]	60 [34/0/26]	**66** [40/0/26]	– –
2	0	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	22 [13/9/0]	31 [26/0/5]	38 [20/18/0]	47 [33/0/14]	55 [55/0/0]	63 [36/0/27]	63 [36/0/27]	**66** [39/0/27]	– –
	1	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	22 [13/9/0]	25 [10/10/5]	40 [22/18/0]	46 [34/0/12]	55 [55/0/0]	62 [40/0/22]	**64** [38/0/26]	– –	– –
	2	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	21 [11/10/0]	30 [20/0/10]	38 [26/0/12]	47 [33/0/14]	55 [55/0/0]	62 [37/0/25]	62 [37/0/25]	**68** [40/0/28]	– –
	3	5 [4/0/1]	9 [7/0/2]	15 [13/0/2]	23 [13/5/5]	31 [26/0/5]	40 [30/0/10]	47 [33/0/14]	54 [28/26/0]	61 [38/0/23]	**64** [41/0/23]	– –	– –
3	0	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	22 [13/0/9]	25 [10/10/5]	40 [22/18/0]	45 [35/0/10]	54 [35/0/19]	61 [42/0/19]	**64** [44/0/20]	– –	– –
	1	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	21 [11/10/0]	30 [25/0/5]	38 [20/18/0]	45 [35/0/10]	54 [35/0/19]	61 [42/0/19]	**64** [37/0/27]	– –	– –
	2	5 [4/0/1]	9 [7/0/2]	13 [4/9/0]	21 [11/10/0]	29 [14/10/5]	38 [20/18/0]	45 [35/0/10]	54 [35/0/19]	61 [42/0/19]	**64** [41/0/23]	– –	– –
	3	5 [4/0/1]	9 [7/0/2]	15 [13/0/2]	25 [15/5/5]	30 [24/0/6]	41 [41/0/0]	46 [32/0/14]	57 [57/0/0]	63 [37/0/26]	63 [37/0/26]	**67** [41/0/26]	– –

D Examples of Input–Output Differences

Table 24: An example of input–output differences for the 8-round Dialga-256 (specifically, 4/2/2 rounds in the forward/middle/backward directions). The DCP is 2^{-49}, and its clustering effect is $2^{-43.679}$. The values are represented in hexadecimal.

Δ_d^{in}	00800000020008000030000000000000
$\Delta_{t_0}^{in}$	00000000000000000000000000000000
$\Delta_{t_1}^{in}$	00000000400000000010000000001800
Δ_d^{out}	080A020A000000000000000000020202
$\Delta_{t_0}^{out}$	00000000000000000000000000000000
$\Delta_{t_1}^{out}$	00000000000000000000000040404800

Table 25: An example of input–output differences for the 9-round Dialga-256 (specifically, 5/2/2 rounds in the forward/middle/backward directions). The DCP is 2^{-91}, and its clustering effect is $2^{-81.221}$. The values are represented in hexadecimal.

Δ_d^{in}	48208000000008000003000D000010002
$\Delta_{t_0}^{in}$	00000000000000000000000000000000
$\Delta_{t_1}^{in}$	00200000000008000000009000000000
Δ_d^{out}	08282028000000000000000000020202
$\Delta_{t_0}^{out}$	00000000000000000000000000000000
$\Delta_{t_1}^{out}$	00000000000000000000000000001000

Table 26: An example of input–output differences for the 10-round Dialga-256 (specifically, 5/2/3 rounds in the forward/middle/backward directions). The DCP is 2^{-96}, and its clustering effect is $2^{-88.082}$. The values are represented in hexadecimal.

Δ_d^{in}	00108404000100002000004004000480
$\Delta_{t_0}^{in}$	00000000000000000000000000000000
$\Delta_{t_1}^{in}$	00008000000000002000000000000080
Δ_d^{out}	40454101040404000048484800040404
$\Delta_{t_0}^{out}$	00000000000000000000000000000000
$\Delta_{t_1}^{out}$	00000000000000000000040000000000

E Results with the UMC 40nm Standard cell library

Table 27: Comparative Evaluation Metrics for the UMC 40nm Process and Library.

		Area optimized					*Latency optimized*				
		Area		Delay	Power	FOM	Area		Delay	Power	FOM
Cipher	Rounds	μm^2	GE	ns	mW	% AES	μm^2	GE	ns	mW	% AES
AES-128	10	56584	103105	50.79	63.59	100.00	86006	156717	10.53	58.09	100.00
AES-192	12	67182	122417	59.40	81.86	150.55	90495	164896	12.66	116.48	241.07
AES-256	14	80054	145870	70.10	115.83	251.38	120415	219414	14.87	123.32	299.79
PRESENT-80	31	9724	17719	24.53	5.97	9.06	60061	109441	3.81	23.07	28.74
PRESENT-128	31	10151	18496	25.49	5.15	8.14	60575	110377	3.82	22.68	28.33
MIDORI-64	16	4996	9103	17.16	2.37	2.52	33691	61390	2.25	11.62	8.55
MIDORI-128	20	12731	23198	21.37	7.95	5.26	85847	156427	2.92	36.26	17.31
PRINCE	12	4266	7773	12.24	1.45	1.10	30027	54714	1.83	6.27	3.75
PRINCE v2	12	4568	8324	14.60	1.65	1.48	32360	58965	1.87	11.06	6.76
Orthros	12	14965	27269	13.66	5.81	2.46	99089	180555	1.78	18.44	5.37
Gleeok-128	12	24883	45341	16.25	9.64	4.85	135652	247180	1.78	22.88	6.66
	10	20100	36624	11.93	5.73	2.11	111193	202612	1.48	15.02	3.63
Gleeok-256	16	87805	159995	22.92	33.84	24.02	315237	574411	2.84	93.63	21.74
	12	63418	115558	16.00	11.56	2.86	229097	417451	2.08	47.84	8.13
SPEEDY-5	5	12007	21879	11.83	2.80	1.02	66357	120912	1.64	9.78	1.75
SPEEDY-6	6	14580	26568	14.76	4.66	2.13	89101	162356	1.93	17.21	3.62
SPEEDY-7	7	17177	31298	17.54	6.59	3.58	95910	174762	2.38	22.82	5.92
ASCON-p12	12	21317	38842	17.84	6.29	3.47	133581	243406	2.01	11.24	1.48
SKINNY-64-192	32	14682	26753	28.37	11.18	9.82	35773	65184	8.57	23.32	65.34
SKINNY-128-128	40	64935	118321	55.63	121.12	208.61	127966	233173	20.80	198.92	676.41
SKINNY-128-384	40	72157	131481	55.63	121.45	209.19	129446	235872	22.45	203.66	747.47
MANTIS-7	16	6292	11465	18.56	2.85	3.27	37242	67860	2.35	8.35	6.42
MANTIS-8	18	7095	12929	20.73	3.65	4.69	41391	75421	2.65	10.63	9.21
BIPBIP-Dec	11	13501	24600	18.04	4.71	14.02	47524	86597	1.62	5.61	7.92
BIPBIP-Enc	11	18146	33064	39.33	9.28	60.29	79854	145506	3.57	18.04	56.15
Aradi	16	14063	25625	31.13	7.06	6.81	70134	127796	3.07	14.81	7.43
Scarf	8	2877	5241	7.54	0.30	0.85	14787	26944	1.06	1.09	2.26
Sonic-256	24	16638	30317	21.38	4.96	1.64	83019	151274	1.98	5.71	0.92
Sonic-512	24	33216	60525	22.16	10.16	1.74	161056	293469	2.02	10.99	0.91
SuperSonic-256	21	16216	29549	21.27	6.75	2.22	77911	141966	2.11	6.58	1.14
SuperSonic-512	21	32432	59096	22.28	14.28	2.46	146185	266372	2.17	12.60	1.12
Twinkle-256	$18\frac{1}{2}$	134473	245031	25.09	53.42	4.61	645493	1176190	3.74	112.77	7.66
QARMAv2-128-128 ($r = 9$)	20	16285	29674	25.18	10.20	7.95	100149	182487	3.27	35.89	19.19
QARMAv2-128-128 ($r = 11$)	24	19548	35619	30.33	14.88	13.97	124534	226921	3.90	53.83	34.32
QARMAv2-128-192 ($r = 13$)	28	22823	41586	35.74	20.67	22.88	147435	268650	4.55	75.74	56.34
QARMAv2-128-256 ($r = 15$)	32	26117	47590	43.23	28.45	38.08	170161	310060	5.20	99.41	84.51
Dialga-256 (Enc only)	20	19015	34649	25.07	9.84	7.63	96986	176724	3.21	29.01	15.22
Dialga-256† (Enc only)	16	15255	27798	21.15	6.05	3.96	80422	146542	2.54	18.08	7.51
Dialga-256-PAC (Enc only)	12	11367	20712	15.11	4.60	2.15	58843	107221	1.97	10.34	3.33
Dialga-256 (Enc+Dec)	20	27683	50443	35.63	18.30	20.19	172927	315101	3.44	54.04	30.39
Dialga-256† (Enc+Dec)	16	22303	40639	27.88	11.32	9.77	142088	258907	2.78	33.92	15.42
Dialga-128 (Enc only)	20	16388	29862	25.02	9.57	7.42	92683	168884	3.15	29.13	15.00
Dialga-128† (Enc only)	16	13204	24060	19.09	5.72	3.38	75996	138477	2.52	18.48	7.62
Dialga-128-PAC (Enc only)	12	10114	18429	15.23	3.19	1.50	57749	105228	1.94	9.95	3.16
Dialga-128 (Enc+Dec)	20	24851	45283	35.18	8.26	9.00	169089	308108	3.38	32.76	18.10
Dialga-128† (Enc+Dec)	16	20184	36779	28.24	5.79	5.07	141107	257120	2.72	28.22	12.55

F Circuits with Unified Encryption and Decryption

Circuits that perform both encryption and decryption are useful in modes of operation that require calls to both the ENC/DEC modules of the underlying block cipher like CBC. Since we try to use Dialga to instantiate the Flat-ΘCB mode which requires both ENC/DEC functions of the block cipher, we also looked to implement the combined circuit in the fully unrolled setting.

Unlike PRINCE and QARMAv2, the core encryption circuit of Dialga is not an involution, and so its ENC/DEC circuit is not as efficient as the encryption-only circuit. However we can perform some optimization to reduce circuit size in the following manner. We notice that in forward and inverse round functions of Dialga, the order of operations is as follows:

$$\begin{aligned}\texttt{ENC}:&\ \mathsf{SubCell} \to \mathsf{ShuffleCell} \to \mathsf{MixColumn} \to \mathsf{KeyAdd}\\ \texttt{DEC}:&\ \mathsf{SubCell} \to \mathsf{KeyAdd} \quad \to \mathsf{MixColumn} \to \mathsf{InvShuffleCell}\end{aligned}$$

This is possible because the SubCell and MixColumn operations in Dialga are involutive. Also, the order of operations in decryption can be obtained by realigning the inverse rounds, i.e., by taking the last SubCell in inverse round i and merging with the first 3 operations KeyAdd, MixColumn, InvShuffleCell of inverse round $i+1$. This shows that a combined circuit that combines both the forward and inverse round can be obtained by inserting two multiplexers in the round function circuit, as shown below.

$$\texttt{ENC+DEC}: \mathsf{SubCell} \to \frac{\mathsf{ShuffleCell}}{\mathsf{KeyAdd}} \to \mathsf{Mux} \to \mathsf{MixColumn} \to \frac{\mathsf{KeyAdd}}{\mathsf{InvShuffleCell}} \to \mathsf{Mux}$$

Note that unlike most SPN constructions, the SubCell and ShuffleCell/InvShuffleCell layers do not commute. This prevents us from doing further optimization by swapping the order of these operations. This would be the most efficient way to implement the circuit if we intend to minimize the area. However since Dialga uses four different round functions the composition of each round is different. It is not too difficult to find the composition of each round by writing down the individual operations of the encryption and decryption in the forward and backward directions. For example for the 20-round Dialga-256 it can be deduced that the r-th round circuit (for $r \in [1, 8]$) is of the form (SC, MC are used as shorthands for SubCell, MixColumn respectively):

$$\mathsf{SC} \to \frac{\pi_{r \bmod 4}}{\oplus MS(R^{\lfloor \frac{r+1}{2} \rfloor}(T_{r \bmod 2}))} \to \mathsf{Mux} \to \mathsf{MC} \to \frac{\oplus R^{\lfloor \frac{r+1}{2} \rfloor}(T_{r+1 \bmod 2})}{\pi^{-1}_{3-r \bmod 4}} \to \mathsf{Mux}$$

For $r \in [11, 19]$ it is of the form:

$$\mathsf{SC} \to \frac{\pi_{r \bmod 4}}{\oplus R^{\lfloor \frac{20-r}{2} \rfloor}(T_{r \bmod 2})} \to \mathsf{Mux} \to \mathsf{MC} \to \frac{\oplus MS(R^{\lfloor \frac{20-r}{2} \rfloor}(T_{r+1 \bmod 2}))}{\pi^{-1}_{3-r \bmod 4}} \to \mathsf{Mux}$$

For the starting and the middle rounds, i.e. $r = 0, 9, 10, 20$, the rounds are slightly different and can be similarly deduced by the writing the operations in order side by side. For example for $r = 0$, we have

$$\frac{\oplus K_0 \oplus K_1 \oplus T_0}{\oplus K_0 \oplus K_1 \oplus MS(T_1)} \to \mathsf{Mux} \to \mathsf{SC} \to \frac{\pi_0}{\oplus MS(T_0)} \to \mathsf{Mux} \to \mathsf{MC} \to \frac{K_1 \oplus T_1}{\pi_3^{-1}} \to \mathsf{Mux}$$

For $r = 9, 10$, we have

$$\mathsf{SC} \to \frac{\pi_1}{\oplus R^5(T_1) \oplus S(R^5(T_0))} \to \mathsf{Mux} \to \mathsf{MC} \to \frac{R_5(T_0)}{\pi_2^{-1}} \to \mathsf{Mux}$$

$$\mathsf{SC} \rightarrow \frac{\pi_2}{\oplus R^5(T_0)} \rightarrow \mathsf{Mux} \rightarrow \mathsf{MC} \rightarrow \frac{\oplus R^5(T_1) \oplus S(R^5(T_0))}{\pi_1^{-1}} \rightarrow \mathsf{Mux}$$

And for $r = 20$ we have

$$\mathsf{SC} \rightarrow \frac{MS(T_1) \oplus K_0 \oplus K_1}{\oplus K_0 \oplus K_1 \oplus T_0} \rightarrow \mathsf{Mux}$$

Note that $R^i(T_j)$ is shorthand for the tweak T_j being updated i times following the path shown in Figure 5, i.e. using the different round functions R_i applied in the correct order.

However, the above circuit adds around 32/40 multiplexers (for the 16/20 round versions) to the critical path of the circuit and is not the best strategy for latency minimization. As far as we can see, the best way to minimize latency is to implement the encryption and decryption circuit in parallel and have one multiplexer in the end, which filters the output of the encryption and decryption paths. This doubles the area of the circuit but limits the total delay to `max`(ENC, DEC) + the delay due to the multiplexer. Table 18 additionally tabulates the results of both the area and delay-optimized ENC/DEC circuit (in the rows shaded in green). Since `QARMAv2` decryption is achieved with very low overhead over the encryption circuit, we can assume that the unified circuit for `QARMAv2` will have approximately the same performance metrics as the encryption-only circuits in Table 18. The table also shows that the unified circuits for 20 round Dialga-256 and Dialga-128 compare reasonably wrt `QARMAv2`-128-256 and `QARMAv2`-128-192. The 16-round circuits are smaller (for the area-optimized circuit) and faster (for the latency-optimized circuit) than the corresponding members of the `QARMAv2` family.

G Round based Circuits

Dialga uses 4 different types of bit-shuffles π_i, $i \in [0, 3]$, and hence four different round functions. As such the round based circuit has to accommodate for this and include a 4:1 multiplexer to filter the outputs of the π_i layers as shown in Figure 15. In this figure the top half (enclosed in a pink box) represents the datapath of the state update. The bottom part in yellow represents the update circuitry of the tweak. Note that each half of the tweak needs to be updated in every alternate round, with the forward round in the first half of the encryption, and with the inverse round function in the second half. Hence the output of the tweak update needs to be filtered by a 3:1 multiplexer, to accommodate for the three cases: forward update, backward update and no update. In case the design employs a 256-bit tweak, this circuit has to be replicated for the upper and lower 128-bit halves of the tweak.

Of course we could also design a 2-round and 4-round unrolled circuit in a similar manner that would need lesser number of multiplexers to filter the outputs if the π_i's. Table 28 tabulates the simulation results of the Dialga-256 and Dialga-128 round based, 2-round and 4-round unrolled circuits. We compare the results with AES-256. We find that both in terms of circuit are and energy consumed Dialga performs reasonably well when compared with AES-256. Note that t rounds of Dialga actually employs the Subcell a total of $t + 1$ times in the data path. Hence the number of cycles needed to compute t rounds is $t + 2$ (one additional cycle for loading the plaintext/tweak) in the round based design. Similarly in the 2-round/4-round designs it is $t/2 + 2$ and $t/4 + 2$ respectively.

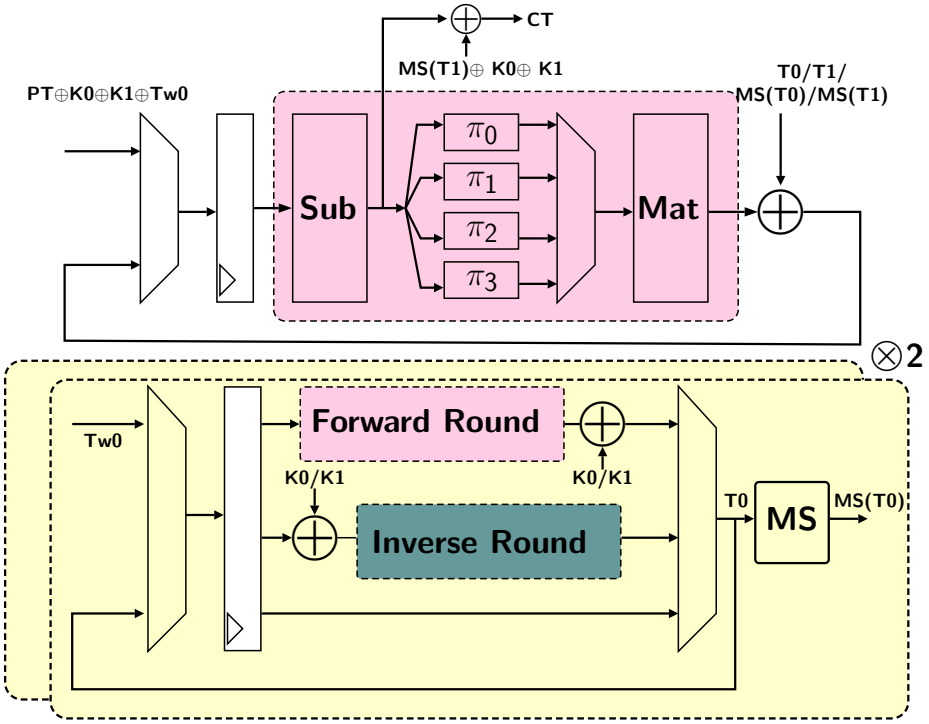

Figure 15: Circuit structure of the Dialga-256 round based design

Table 28: Simulation Results for Nangate 15nm library.

	Rounds	Area		Latency	Power	Cycles	Energy
		(μm^2)	(GE)	(ps)	(μW)		(pJ)
Round based Circuits							
AES-256	14	2205	11215	524	99.5	15	149.25
Dialga-256†	16	3254	16549	1153	182.7	18	328.86
Dialga-256	20	3338	16979	1093	177.5	22	390.48
Dialga-128†	16	2130	10836	831	91.3	18	164.30
Dialga-128	20	2183	11106	928	113.2	22	249.04
2-round Unrolled							
Dialga-256†	16	3424	17414	762	297.9	10	297.93
Dialga-256	20	3511	17856	1043	359.5	12	431.40
Dialga-128†	16	2341	11905	718	240.0	10	239.96
Dialga-128	20	2418	12298	652	286.5	12	343.80
4-round Unrolled							
Dialga-256†	16	4858	24708	644	710.5	6	426.28
Dialga-256	20	5039	25629	641	891.3	7	623.91
Dialga-128†	16	3199	16270	578	509.8	6	305.87
Dialga-128	20	3267	16617	833	479.4	7	335.58
3-Share Threshold Implementation							
Dialga-256†	16	16638	84627	797	774.2	35	2709.83
Dialga-256	20	16801	85457	1008	799.7	43	3438.71
Dialga-128†	16	10279	52281	947	493.6	35	1727.72
Dialga-128	20	10396	52878	1045	493.6	43	2122.48

H Masked Circuits

The S-box used in Dialga is the same as Sb_0 used in Midori-128. The S-box belongs to the cubic class $\mathcal{C}_{266}$ described in [BNN+12], (same as the PRESENT S-box) and can this be decomposed as the composition of two quadratic S-boxes. This is particularly useful because this allows for a three share threshold implementation of Sb_0 over two clock cycles. The decomposition of Sb_0 we can use for this purpose was outlined in [DO23]. It decomposes Sb_0 as:

$$\mathsf{Sb}_0 = A_3 \circ Q_2 \circ A_2 \circ Q_1 \circ A_1,$$

where A_1, A_2, A_3 are affine permutations over $\{0,1\}^4$ and $Q_1 \in \mathcal{Q}_{294}$ quadratic class and $Q_2 \in \mathcal{Q}_{299}$ quadratic class. These quadratic classes are useful, because the threshold implementation of the S-boxes in these classes can be obtained by the direct sharing technique. For explicit algebraic expressions for A_i, A_2, A_3, Q_1, Q_2 and the corresponding circuit diagrams, please see [DO23, Appendix B] and [DO23, Figure 5]. Using this decomposition we implemented a 3-share threshold implementation of Dialga-256 and Dialga-128 in which each round is executed in two cycles. The results are presented in Table 28.

I Test Vectors

I.1 20-round Variants

Following are the test vectors for Dialga-256 and Dialga-128 for 20 rounds.

I.1.1 Dialga-256

1	Plaintext	00112233445566778899aabbccddeeff
	Key	00112233445566778899aabbccddeeff 11223344556677899aabbccddeeff00
	Tweak	2233445566778899aabbccddeeff1122 33445566778899aabbccddeeff001100
	Ciphertext	e129ff0920371753db65532540d06881
2	Plaintext	0123456789abcdef0123456789abcdef
	Key	fedcba9876543210fedcba9876543210 fedcba9876543210fedcba9876543210
	Tweak	00001111222233334444555566667777 88889999aaaabbbbccccddddeeeeffff
	Ciphertext	b30b9d8a8d6dd98d235a4aaeda5b06ca

I.1.2 Dialga-128

1	Plaintext	00112233445566778899aabbccddeeff
	Key	00112233445566778899aabbccddeeff 11223344556677899aabbccddeeff00
	Tweak	2233445566778899aabbccddee00ff11
	Ciphertext	a4a1ea948919d8996e13b1b365bb0ce6
2	Plaintext	0123456789abcdef0123456789abcdef
	Key	fedcba9876543210fedcba9876543210 fedcba9876543210fedcba9876543210
	Tweak	00001111222233334444555566668888
	Ciphertext	dc355a6376d9617723efb9a98c1b4864

I.2 16-round Variants

Following are the test vectors for Dialga-256 and Dialga-128 for 16 rounds.

I.2.1 Dialga-256†

1	Plaintext	00112233445566778899aabbccddeeff
	Key	00112233445566778899aabbccddeeff 11223344556677899aabbccddeeff00
	Tweak	2233445566778899aabbccddeeff1122 33445566778899aabbccddeeff001100
	Ciphertext	833367ce5fd49a7b1f0dd9ec62720018
2	Plaintext	0123456789abcdef0123456789abcdef
	Key	fedcba9876543210fedcba9876543210 fedcba9876543210fedcba9876543210
	Tweak	00001111222233334444555566667777 88889999aaaabbbbccccddddeeeeffff
	Ciphertext	0f9991a2b1f311c8c786ad0434edc15a

I.2.2 Dialga-128†

1	Plaintext	00112233445566778899aabbccddeeff
	Key	00112233445566778899aabbccddeeff 112233445566778899aabbccddeeff00
	Tweak	2233445566778899aabbccddee00ff11
	Ciphertext	838407143af9a876fbdc6be378e9045b
2	Plaintext	0123456789abcdef0123456789abcdef
	Key	fedcba9876543210fedcba9876543210 fedcba9876543210fedcba9876543210
	Tweak	00001111222233334444555566668888
	Ciphertext	a16812d9738333d238c23e67ac20ef16

I.3 12-round Variants

The following are the test vectors for Dialga-128-PA and Dialga-256-PA.

I.3.1 Dialga-128-PA

1	Plaintext	00112233445566778899aabbccddeeff
	Key	00112233445566778899aabbccddeeff 112233445566778899aabbccddeeff00
	Tweak	2233445566778899aabbccddee00ff11
	Ciphertext	3fc5b21ac977463aab8355019d4e4e33
2	Plaintext	0123456789abcdef0123456789abcdef
	Key	fedcba9876543210fedcba9876543210 fedcba9876543210fedcba9876543210
	Tweak	00001111222233334444555566668888
	Ciphertext	00e58d83ae2eb59511a7445dfb850634

I.3.2 Dialga-256-PA

1	Plaintext	00112233445566778899aabbccddeeff
	Key	00112233445566778899aabbccddeeff 112233445566778899aabbccddeeff00
	Tweak	2233445566778899aabbccddee00ff11
	Ciphertext	c300d8c0d7cb5e56f744b4b635a74cb4
2	Plaintext	0123456789abcdef0123456789abcdef
	Key	fedcba9876543210fedcba9876543210 fedcba9876543210fedcba9876543210
	Tweak	00001111222233334444555566668888
	Ciphertext	7db68346027c95d670e1fd46c11b9cf9

I.4 Per-round Test Vectors

Following are the per-round test vectors for Dialga-256† and Dialga-128† for 16 rounds.

I.4.1 Dialga-256†

Plaintext	00112233445566778899aabbccddeeff
Key	00112233445566778899aabbccddeeff
	112233445566778899aabbccddeeff00
Tweak	2233445566778899aabbccddeeff1122
	33445566778899aabbccddeeff001100
r_1	532ea66649c5b3c29bf7207bed25fb00
r_2	68f551577d90d741e47ef1c492b309b1
r_3	d2f4cd4495ca249118faa7ef77b29488
r_4	019584fc6aadc3dc7159deaf8b9c598d
r_5	a0f832285c988302a78436058aa1b5ea
r_6	28bdf5cffe3cf26b495faf0486e5f6a6
r_7	300aa5780bd7749acecd0bbc5e62300f
r_8	dc6ddc2f5b2bf7778d678b646e6be7be
r_9	c2d77bf88079b4e40658ad332701b32e
r_{10}	76946ec3a4e948cb34cb8200bf8aef5c
r_{11}	3ee9efed8d0d12fe76509075814d86ad
r_{12}	2f4f205788c4e36cd129048945f9abee
r_{13}	8432386053953d7a5b440167f1b550ef
r_{14}	17c2edb3182d494b264a9b154d3952f7
r_{15}	a15eaf8e0aa0d125c9714c300d5b8436
r_{16}	4daa1d40e36de6bdda58801d83a4aa6b
Ciphertext	833367ce5fd49a7b1f0dd9ec62720018

I.4.2 Dialga-128†

Plaintext	00112233445566778899aabbccddeeff
Key	00112233445566778899aabbccddeeff
	112233445566778899aabbccddeeff00
Tweak	2233445566778899aabbccddee00ff11
r_1	9f95f3ff7a092a2c465dfdf31225ea00
r_2	98871f6b568e38c69b2df2b8fecc46c4
r_3	1d02f8badc70a734a02946d7584c6699
r_4	0924e3460275180560387c9d89ccaef7
r_5	7df1476adedc82fe587839dc0e43f31c
r_6	dd68245888f9cbed8c05065ddcffc56b
r_7	9da77dade1660eaf5a2877126d7bdeeb
r_8	4c31411fc08dd78da9c94db8175ca087
r_9	52ffe58da193d2b26faaa208042c1dfe
r_{10}	d7beb80383bb057a8a470b61e6f1cd19
r_{11}	d0f79a0ce3b4f519c8b06141af26a4ca
r_{12}	ea549d58f095e1159158679953b8be8a
r_{13}	9ef0091d1fd39ce3f836b8d14d5040eb
r_{14}	efef7cf1a11eae91f1ed0ac3d8773621
r_{15}	87e87173805a0eac97de2f230432bcd0
r_{16}	92a33e3c3115979441131a892119bed7
Ciphertext	838407143af9a876fbdc6be378e9045b